

Practical Workbook
CS-301
Microprocessor Based System Design



Name : _____
Year : _____
Batch : _____
Roll No : _____
Department: _____

Department of Computer & Information Systems Engineering
NED University of Engineering & Technology

Practical Workbook
CS-301
Microprocessor Based System Design



Prepared by:

Syeda Ramish Fatima

Revised in:

April 2024

Revision 3

Department of Computer & Information Systems Engineering
NED University of Engineering & Technology

Introduction

This workbook has been compiled to assist the conduct of practical classes for CS-301 Microprocessor Based System Design. Practical work relevant to this course aims at providing students a chance to interact with a microcontroller kit. These kits can be used with additional peripherals not provided on board and implementation of many microcontroller based system can be easily learned through a series of step-wise lab sessions. Microcontroller programs can be easily written using embedded C and compiled to a hex code which can be run by the controller and for that Atmel studio 7 is chosen. To download the hex code on flash embedded in microcontroller, Khazama Programmer software is used. Additionally, practical work in this workbook also includes simulation based labs to learn basic interfacing of microcontrollers with different peripherals and the simulation software is Proteus.

Course Profile of CS-301 Microprocessor Based System Design lays down the following Course Learning Outcome:

“Practice interfacing microprocessor with other system components (P3, PLO-3)”

All lab sessions of this workbook have been designed to assist the achievement of the above CLO. A rubric to evaluate student performance has been provided at the end of the workbook.

Lab session 01 introduces the students to embedded C programming and Atmel Studio 7 IDE. Lab 02 allows the students to explore AVR Microcontroller architecture and AVR board features. Lab sessions 3 through 6 cover the use of basic AVR board peripherals like LED bank, switches and Seven Segment Displays through ATmega162 GPIO's and learning the basics of C operations on data received from GPIO's. Lab 06 assists in learning about timers and counters.

Lab session 7 and 8 helps in learning to interface LCD's and keypad. Lab session 9 is related to interfacing ADC's.

Lab sessions 10 through 12 covers interfacing microcontroller with different devices using serial communication. Lab sessions 13 and 14 are designed to help students learn interfacing microcontrollers using simulation software. A Complex Engineering Activity Based Project (open-ended lab) will also be assigned to the students (Lab session 15) which will be evaluated in the last week.

Contents

Lab Session No.	Title	Page No.	Teacher's Signature	Date
1	Explore Atmel Studio 7 IDE for programming AVR microcontrollers	1		
2	Explore AVR microcontroller architecture and microcontroller development board	7		
3	Practice using AVR microcontroller GPIO registers by reading data from switches and writing the results to the LEDs.	13		
4	Practice different C operations on data received from AVR microcontroller GPIOs and display results on LED bank	17		
5	Practice on-board seven segment display options by implementing driver algorithms	23		
6	Practice using Timers and Counters in microcontroller	29		
7	Practice LCD interfacing with microcontroller	35		
8	Practice keypad interfacing with microcontroller	43		
9	Practice ADC interfacing with microcontroller	47		
10	Explore Serial Communication and AVR microcontroller interfacing with devices through RS-232(UART)	55		
11	Practice microcontroller interfacing with devices through RS-232(UART)	61		
12	Practice microcontroller interfacing with devices through I ² C protocol	67		
13	Practice motor driver interfacing with microcontroller using Proteus	75		
14	Practice isolating voltage levels between circuits through opto-isolator and microcontroller using Proteus	81		
15	Open Ended Lab	89		
	Grading Rubric Sheets			

Lab Session 01

Explore Atmel Studio 7 IDE for programming AVR microcontrollers

Embedded C programming

Embedded C is a set of language extensions for the C programming language by the C Standards Committee to address commonality issues that exist between C extensions for different embedded systems.

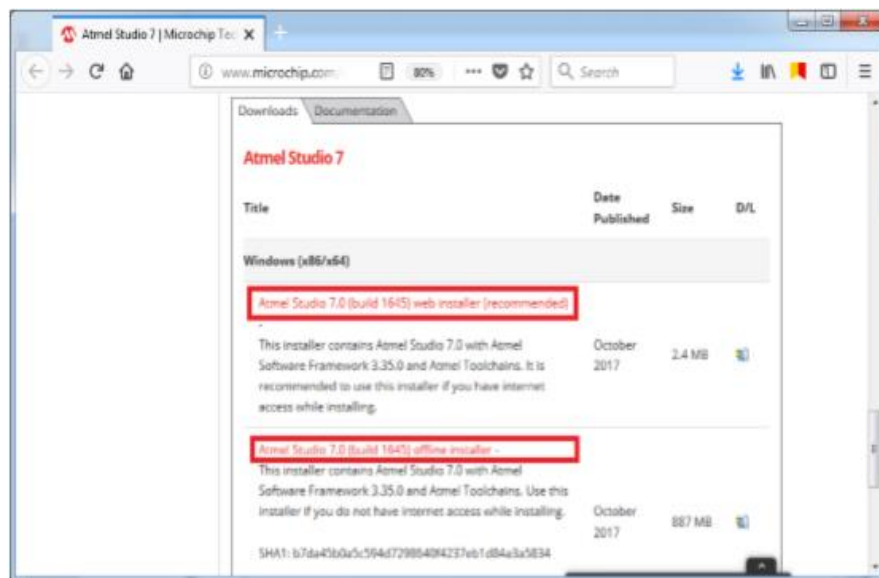
Historically, embedded C programming requires nonstandard extensions to the C language in order to support exotic features such as fixed-point arithmetic, multiple distinct memory banks, and basic I/O operations. In 2008, the C Standards Committee extended the C language to address these issues by providing a common standard for all implementations to adhere to. It includes a number of features not available in normal C, such as fixed-point arithmetic, named address spaces and basic I/O hardware addressing. Embedded C uses most of the syntax and semantics of standard C, e.g., main() function, variable definition, datatype declaration, conditional statements (if, switch case), loops (while, for), functions, arrays and strings, structures and union, bit operations, macros, etc.

In this lab we are going to explore ATMEL STUDIO 7 to understand how to write, compile and trace our C code targeted for a microcontroller.

Downloading and installing Atmel Studio

Download the newest version of Atmel Studio from the Microchip website:

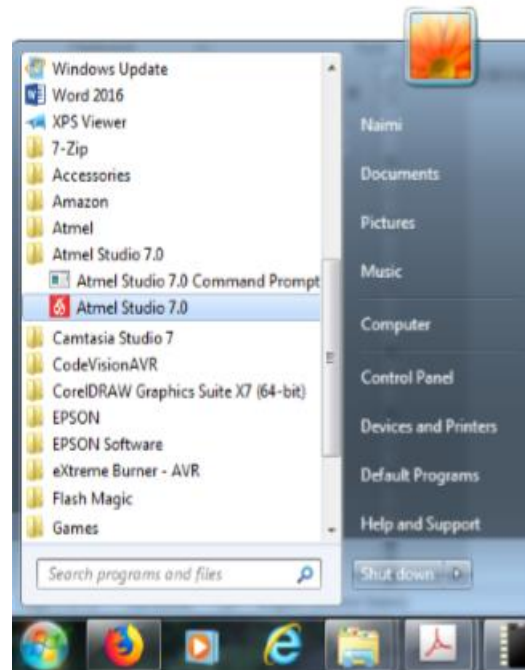
<http://www.microchip.com/>



Run the downloaded program to install the Atmel Studio IDE.

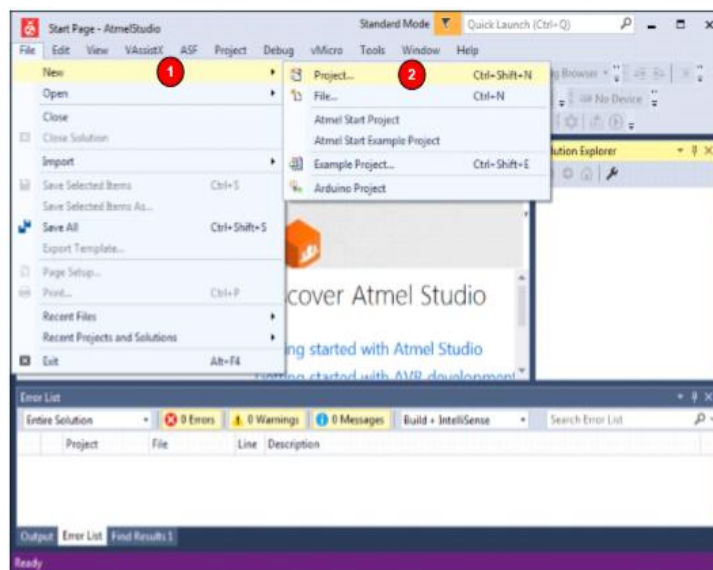
Opening Atmel Studio

Go to the Start menu and open Atmel Studio.



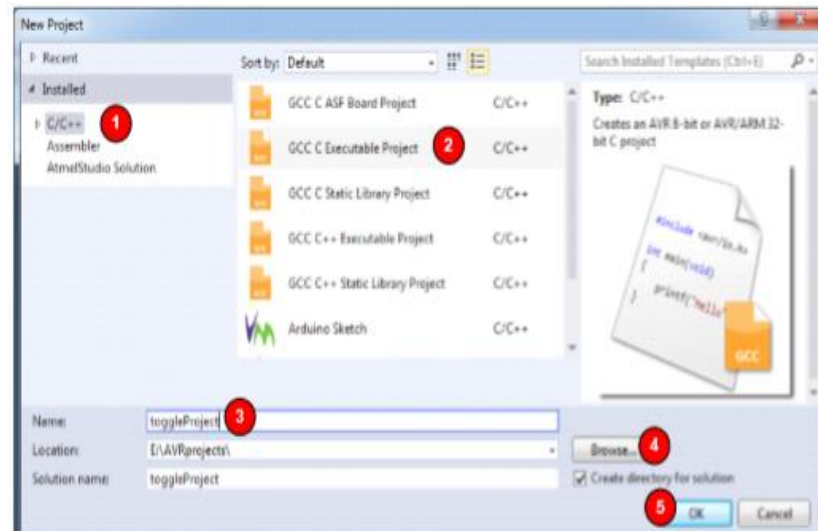
Creating the first project

1. Go to the File menu. Choose New and then Project.



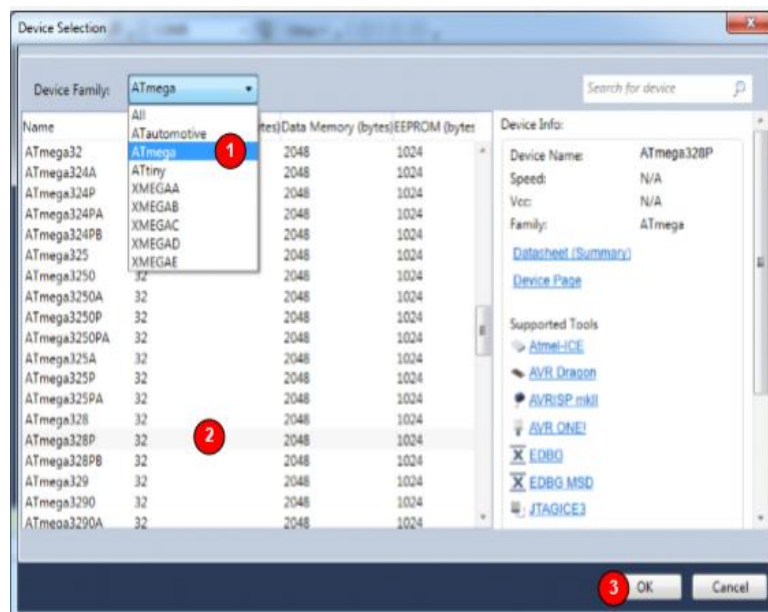
2. In the opened dialog:

- a. Choose C/C++.
- b. Select GCC C Executable Project.
- c. Name the project as toggleProject.
- d. Choose the path where you like to save the project by clicking on the Browse button.
- e. Press OK.

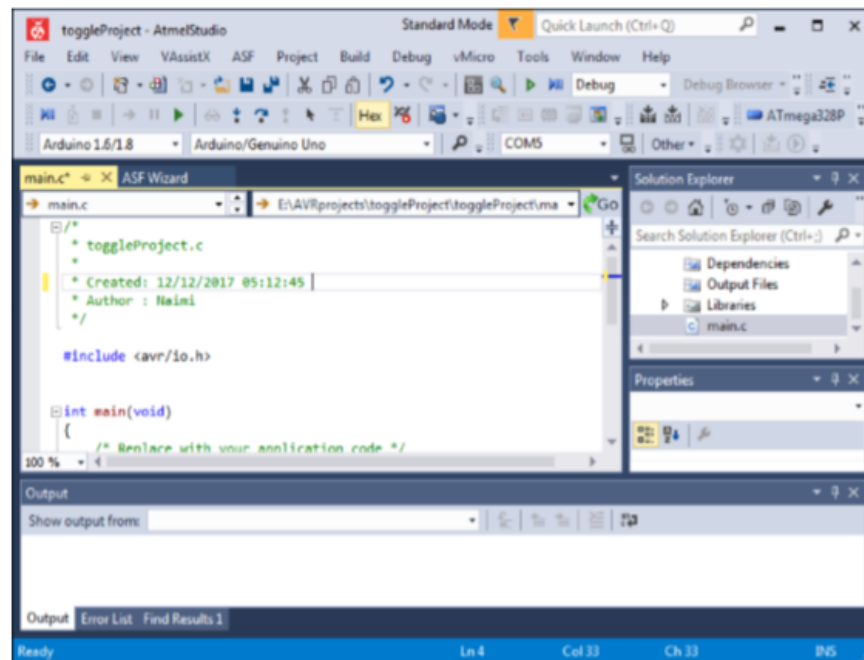


3. In the Device Selection dialog

- a. Select ATmega as the Device family.
- b. Choose ATmega162 (or any other Chips you want to use)
- c. Select OK.



The compiler automatically makes the toggleProject and adds a C file to it.



Writing the first C program

Type the following program

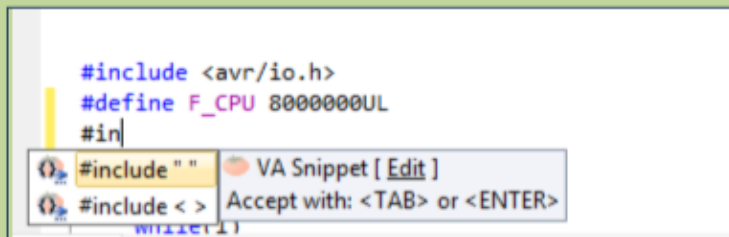
```
#include <avr/io.h>
#define F_CPU 16000000UL
#include "util/delay.h"

int main(void)
{
    DDRB = 0xFF; //make port B as output port

    while(1)
    {
        PORTB = 0xFF; //make all pins HIGH
        _delay_ms(1000); //wait 1 sec
        PORTB = 0x00; //make all pins LOW
        _delay_ms(1000); //wait 1 sec
    }
}
```


Note: Auto Complete

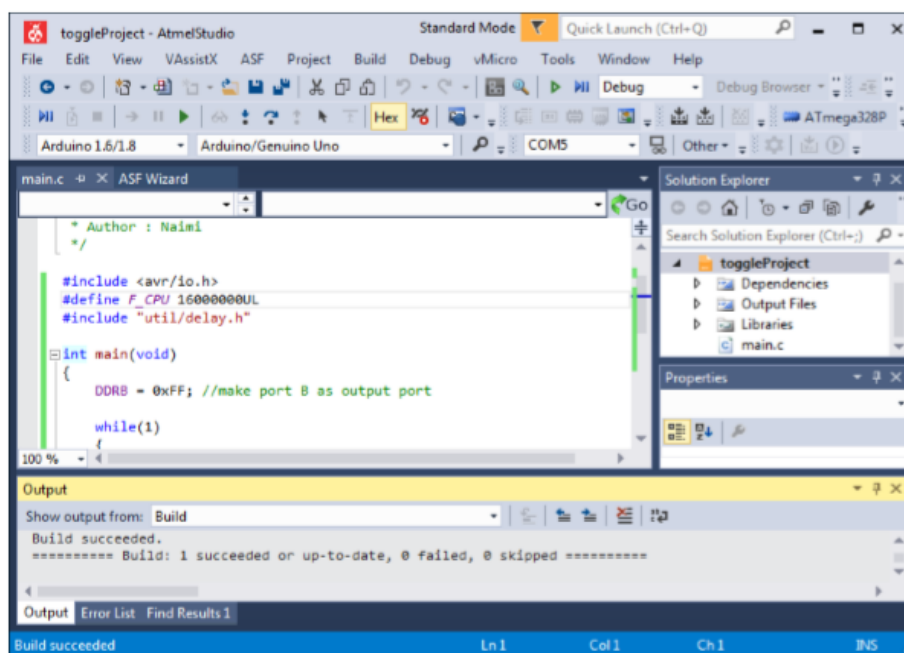
The editor knows the available variables and functions. For example, if you type the first few letters of `#include`, e.g. `#in`, the following list appears and you can choose **#include** from the list, by pressing the **Enter** button instead of typing **#include**.



While editing the program, if you press **Ctrl+Space**, the auto complete list appears, and you can choose between the available choices.

Compiling

Press **F7** to compile, or choose **Build Solution** from the **Build** menu. The results of compile are shown in the **Output** window.



Exercise

1. Write the program given in example, follow the steps to compile and build it and attach the screenshots of output and hex file.

Lab Session 02

Exploring AVR microcontroller architecture and microcontroller development board

Microcontrollers are power full development units providing cost effective application development with high precision as Microcontroller ICs have peripheral modules are integrated in hardware with the processing unit that makes the Microcontrollers muscular and we have a huge variety of Microcontrollers within a family to select the best suited for a particular application domain.

AVR Microcontroller and RISC Architecture:

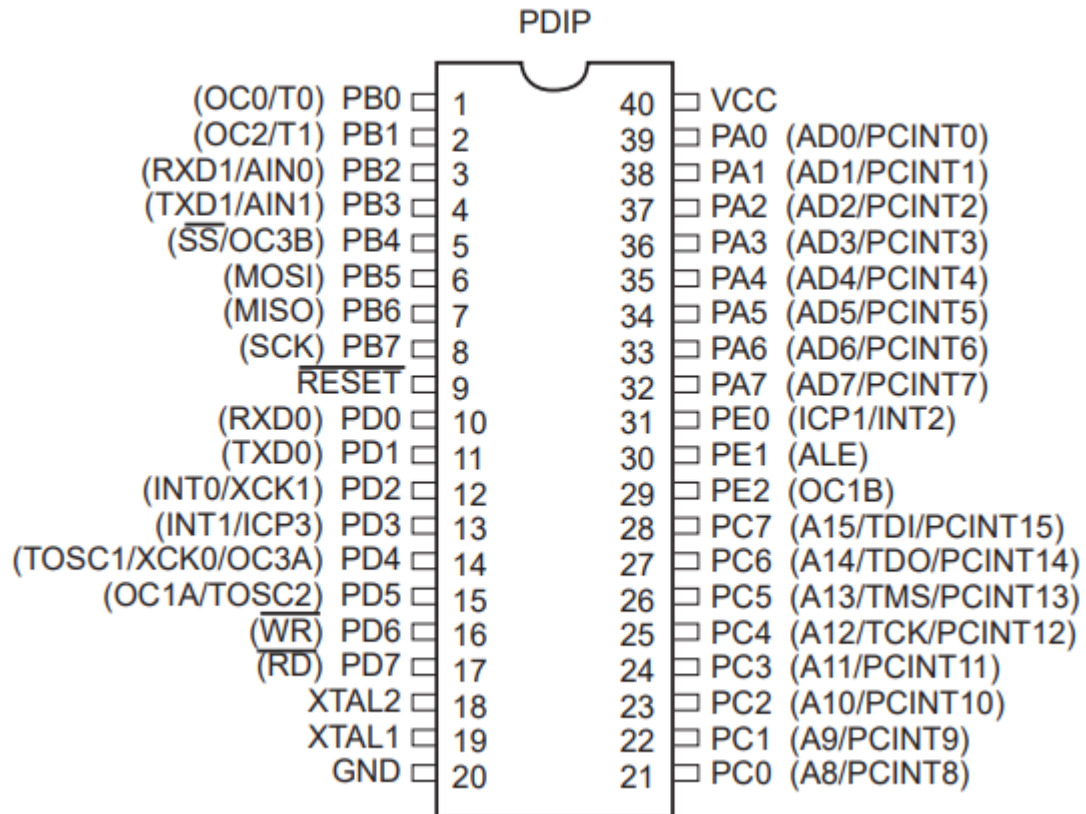
AVR Microcontroller architecture was proposed by Alf-Egil Bogen and Vegard Wollan from Norwegian Institute of Technology (NTH). In 1996 ATMEL launched the first AVR controller. AVR Microcontrollers are one of the first Microcontroller families providing on-chip flash memory. AVR Microcontrollers are based on enhanced RISC architecture providing power full instruction set along with 32 general purpose register directly accessible by ALU, this leads to execution of instructions in a single cycle as ALU can access two registers independently for a single instruction.

ATMega162 Microcontroller

ATMeag162 is an 8 bit microcontroller with following Peripherals and Special Microcontroller features available which we are going to explore in further lab sessions.

- Two 8-bit Timer/Counters with Separate Prescalers and Compare Modes
- Two 16-bit Timer/Counters with Separate Prescalers, Compare Modes, and Capture Modes
- Real Time Counter with Separate Oscillator
- Six PWM Channels
- Dual Programmable Serial USARTs
- Master/Slave SPI Serial Interface
- Programmable Watchdog Timer with Separate On-chip Oscillator
- On-chip Analog Comparator
- Power-on Reset and Programmable Brown-out Detection
- Internal Calibrated RC Oscillator
- External and Internal Interrupt Sources
- Five Sleep Modes: Idle, Power-save, Power-down, Standby, and Extended Standby

Pin diagram of ATmega16 (DIP Package)



AVR Family

AVR TINY

- 8 Bit
- Small and powerful
- General Purpose + Customized
- 0.5 – 8KB Flash
- 6-32 pins
- Up to 20MHz

AVR MEGA

- 8 Bit
- More peripherals and options
- General Purpose + Customized
- 4-256KB Flash
- 28-100-pins
- Up to 20 MHz

AVR XMEGA

- 8/16 Bit
- Xtreme performance
- General Purpose
- 16-384KB Flash
- 44-100 pins
- Up to 32 MHz

AVR ATMEGA

- When your designs need some extra muscle, you need the megaAVR.
- Ideal for applications requiring large amounts of code, the megaAVR offers substantial program and data memories with performance up to 20 MIPS.
- Innovative picoPower technology minimizes power consumption.
- All megaAVRs offer self-programmability for fast, secure, cost-effective in-circuit upgrades.
- Based on industry-leading, proven technology, the megaAVR family offers our widest selection of devices in terms of:
 - memories,
 - pin counts and peripherals.
- Choose from general-purpose devices to models with specialized peripherals like USB, or LCD controllers, or CAN and Power Stage Controllers.
- It's really easy to find the perfect fit for your project in the megaAVR product family.

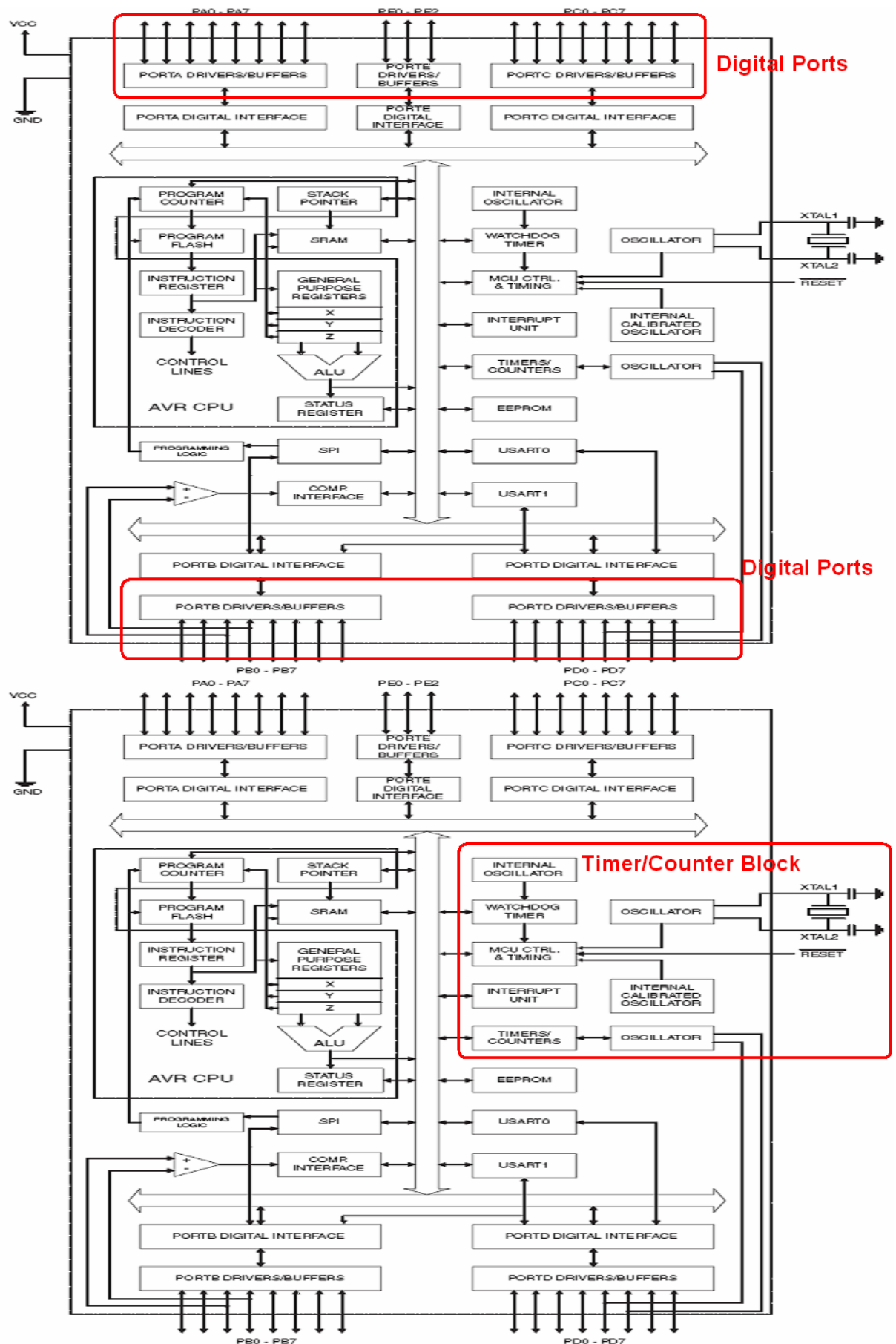
Key features

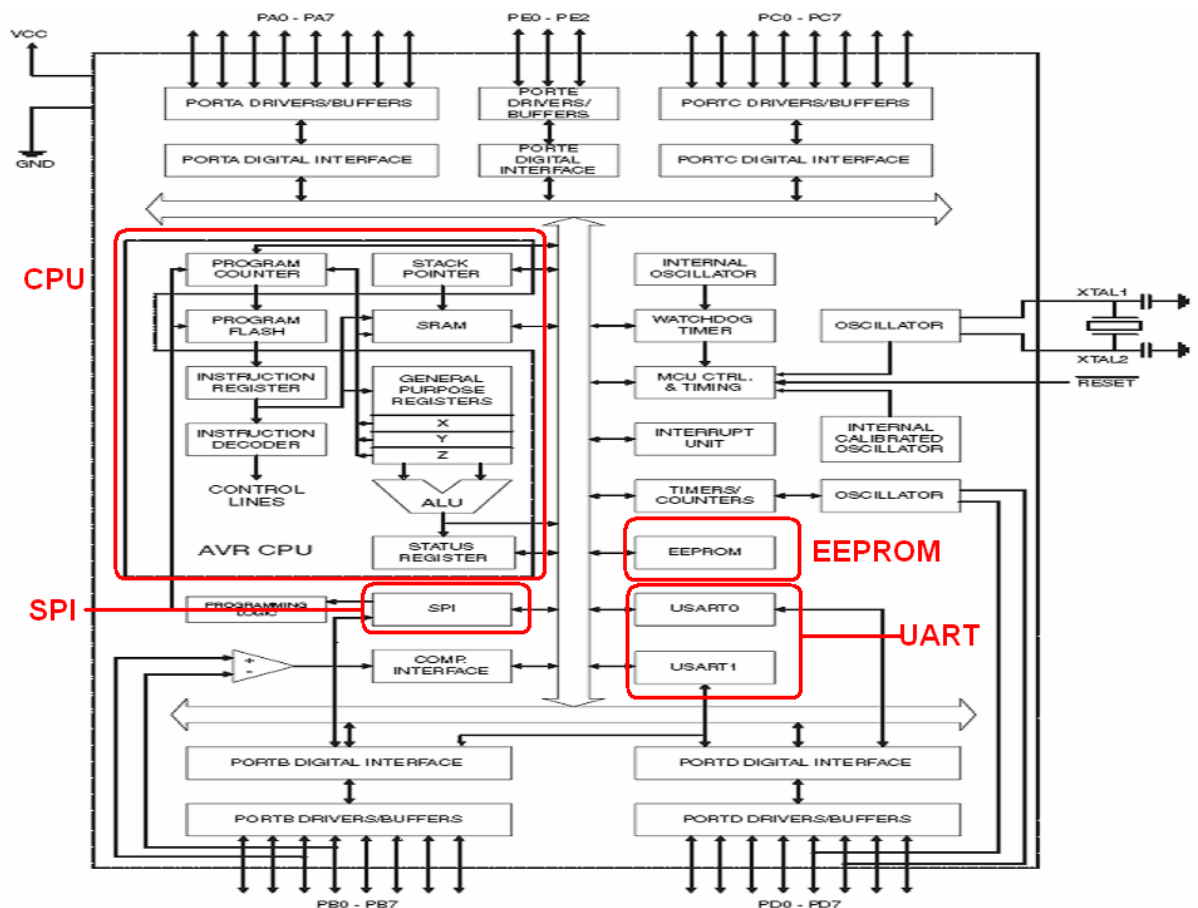
- Broad family — megaAVR offers widest selection of devices in terms of memories, pin counts and peripherals, enabling reuse of code and knowledge across projects.
- picoPower technology — Selected megaAVR features ultra low power consumption and individually selectable low-power sleep modes that make it ideal for battery-powered applications.
- High integration — The megaAVR features on-chip flash, SRAM, internal EEPROM, SPI, TWI, and USART, USB, CAN, and LIN, watchdog timer, a choice of internal or external precision oscillator, and general purpose I/O pins, simplifying your design and reducing bill-of-materials.
- Analog functions — Advanced analog capabilities, such as ADC, DAC, built-in temperature sensor and internal voltage reference, brown out detector, a fast analog comparator and a programmable analog gain amplifier. The high level of integration allows designs with fewer external analog components.
- Rapid development — megaAVR microcontrollers speed development with powerful in-system programming and on-chip debug. In addition, in-system programming simplifies production line programming and field upgrades.

Atmega162

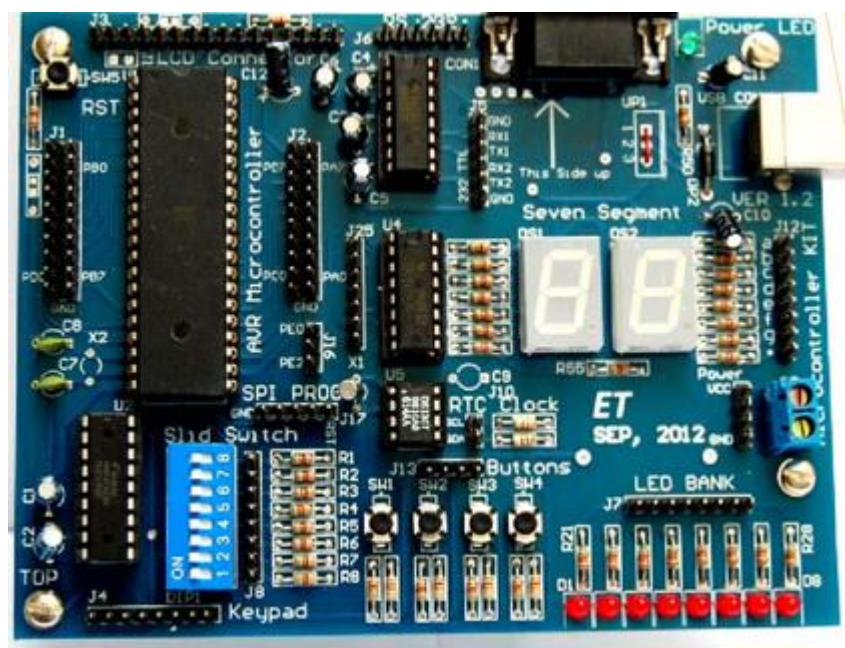
Salient Feature:

- 16K bytes of In-System Programmable Flash
- 512 bytes EEPROM, 1K bytes SRAM
- 35 general purpose I/O lines,
- 32 general purpose working registers,
- Four flexible Timer/Counters with compare modes,
- internal and external interrupts,
- two serial programmable USARTs,
- a programmable Watchdog Timer with Internal Oscillator,
- An SPI port,
- Five software selectable power saving modes.

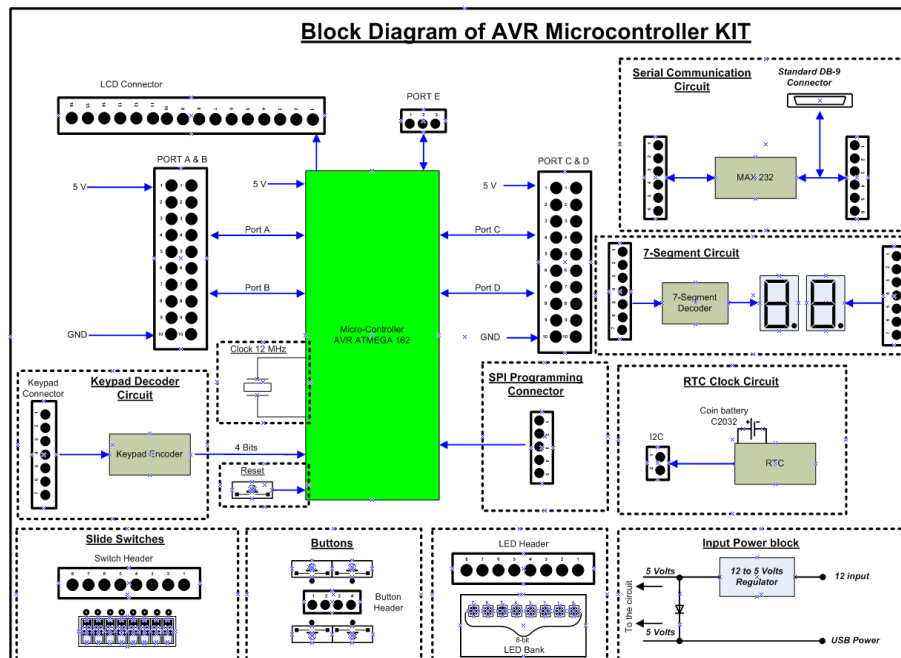




- You need following to start experimenting the AVR:
 - Software Requirements:
 - AVR Studio 7 or any other latest version
 - SPI Programmer (USB Programmer)
 - Hardware Requirements:
 - The AVR Controller kit
 - SPI Programmer kit (USB Based)



Block diagram



Development cycle of avr

- Firmware development cycle of AVR comprises of following steps:
 - Step 01:
 - Create project in AVR Studio
 - Write C-Code in AVR studio
 - Compile the C-Code for error and warning removal
 - Simulate if required
 - Create Hex File
 - Step 02:
 - Connect your target hardware
 - Connect your ISP programmer
 - Step 03:
 - Download hex file to target hardware

Exercise

1. Write down your observations in few lines.

Lab Session 03

Practice using AVR microcontroller GPIO registers by reading data from switches and writing the results to the LEDs

Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC

GPIO's

GPIO stands for general purpose input output. Microcontrollers perform various internal operations and keep final and intermediate results in different registers. Similarly while interacting with external data and command sources they need to store data coming from outside so that they can operate the data or command received. For this purpose Microcontrollers use *GPIO* registers which host the data output or input to or from the external world.

For accessing GPIOs externally Microcontrollers provide pin outs that can be connected to I/O sources. These pins are mapped to GPIO headers on the development board for easy connection.

ATmega162 GPIOs

ATmega162 AVR microcontroller has 4 eight bit and 1 three bit wide GPIOs, thereby hosting 35 GPIO locations. These registers are designated with a suffix A, B, C, D and E.

Associated Registers

Microcontroller uses the following registers to configure the behavior of GPIOs and data storage, the suffix 'x' following the register type must be replaced, in code, with A, B, C, D or E as per your task.

DDRx: configures GPIO as input or output. Writing a 1 to any location of this register will configure the corresponding GPIO pin of MCU as output, while a 0 causes input configuration.

E.g. `DDRA=0x00;` `//this command makes port A an input port`

PINx: used for reading a GPIO port register. This register contains data of a GPIO port that can be retrieved as follows.

E.g. `char y=PINA;` `//here data stored in port A is written in character y;`

PORTx: Used for writing to a GPIO port. Microcontroller writes to a GPIO port by updating this register with the data to be written.

E.g. `PORTA=0x55;` `//here 0x55 is written to port A.`

Program

```
#include<avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>

int main(void)
{
    DDRA=0b11111111; // set port A as output
    DDRB=0b00000000; //set port B as input
    char y;
    while(1)
    {
        y=PINB; //Read input from port A
        PORTA=y;
    } // while ends
} //main ends
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a) Connect 8 pins of PORTA at header J2-AVR with LED bank such that LSB of PORT is connected to right most LED. Connect 4 pins of PORTB with switch bank (J13) and remaining 4 pins to slide switch (J8).

Change the switch state and see the results on the LED

[illegible]

1. Write down the statements to configure port A as output and write 0x55 to it.

2. Write a simple code to blink LEDs in alternate fashion.

[illegible]

- Design a free running counter, using GPIOs, which increments by single digit only when dip switch is pressed.

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 04

Practice different C operations on data received from AVR GPIOs and display results on LED bank

Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC

Introduction

- Data acquired from GPIOs is operated to get final results of desired task.
- C language provides following types of operators.
- Arithmetic operators
- Logical operators
- Assignment operators
- Bitwise operators

Arithmetic operators

- Following are the arithmetic operators supported in C language.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
++	Increment
--	Decrement

Example

Let P=10, Q=20 and port A is used as output;
P0=P+Q; //30 is received at port 0
P0= P++; //11 is received at port 0

Logical operators

Following are the logical operators supported in C language.

Operator	Description
==	Equal
!=	not equal
>	greater than

<	less than
>=	greater than or equal
<=	less than or equal
	logical OR
&&	logical AND
!	NOT

Example:

(P==Q) //returns TRUE if P is equal to Q

(P>=Q) //returns TRUE if P is greater than or equal to Q

(P>5 && Q!=10) //returns TRUE if P is greater than 5 and Q is not equal to 10

Bitwise operators

Following are the Bitwise operators supported in C language.

Operator	Description
&	AND
	OR
^	XOR
~	Invert
<<	shift left
>>	shift right

Example:

Let A=0b0000 1111, B= 0b0101 0101

PORTA= A&B; //0b0000 0101 is received at port A

PORTA= A^B; //0101 1010 is received at port A

PORTA= B>>2; //0001 0101 is received at port A (contents of B is shifted right 2 times).

Program

```
#include<avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>
int main(void)
{
    DDRA=0b00000000; // set port A as input
    DDRB=0b00000000; //set port B as input
    DDRC=0b11111111; //set port C as output
    char P,Q,R;
    while(1)
    {
        P=PINA; //Read input from port A
        Q=PINB; //Read input from port B

        R=P+Q;
        PORTC=R;
        _delay_ms_(200);
        R=P++;
        PORTC=R;
        _delay_ms_(200);
        R=P&Q;
        PORTC=R;
        _delay_ms_(200);
        R=P^Q;
        PORTC=R;
        _delay_ms_(200);
        R=P>>2;
        PORTC=R;
        _delay_ms_(200);
    }
}
```

```
R=~Q;
PORTC=R;
_delay_ms_(200);
R=P&(~Q);
PORTC=R;
_delay_ms_(200);
} // while ends
} //main ends
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a) Connect 8 pins of PORTC at header J1-AVR with LED bank such that LSB of PORT is connected to right most LED. Connect 4 pins of PORTA with switch bank (J13) and 8 pins of Port B to slide switch (J8).
 - b) Take the USB cable and connect the USB connector to the development board USB B type
 - c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.
7. Launch Khazama programming software and select ATmega162 from the drop down list.
8. Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.
9. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

- 3- Write a small code for AVR controller that performs the following operation in sequence. Considering A = 0x0A and B is a free running counter :
- Glows LED 0 if and only if A is equals to B
 - Glows LED 1 if and only if A is not equals to B
 - Glows LED 2 if and only if A is greater than B
 - Glows LED 3 if and only if A is less than B

[illegible]

- 4- Design a 4 operation (add, sub, AND, OR) calculator with following specifications. Take two 3-bit operands from port A and operator from port B as input. Display result on LED Bank.

[illegible]

Lab Session 05

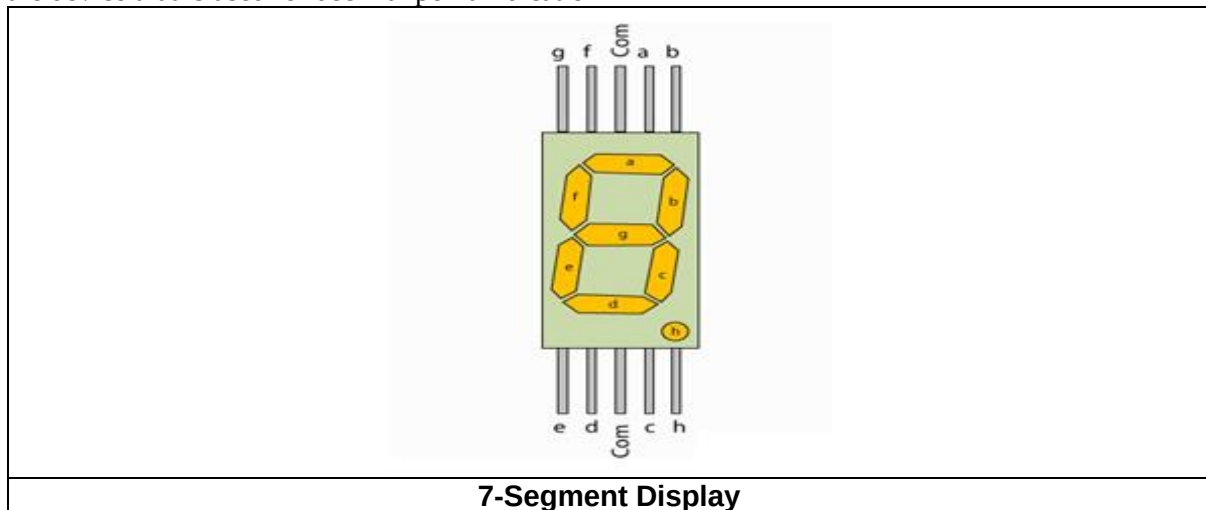
Practice on-board seven segment display options by implementing driver algorithms

Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e. AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC

Introduction

Seven segments is the most basic electronic device that can display numerical digits 0-9. The display is created by a combination of seven LEDs that can be ON/OFF individually. There is one more LED in the device that is used for decimal point indication.



7-segement display is a ten pin device, in which 8 pins are the single terminals of eight LEDs, while the other terminal of all LEDs is connected together and named as common(Com). Based on common pin connection there are two types of Seven Segment Display configuration.

Common Anode

In this configuration Common pin is internally short with Vcc, to turn an LED on the respective segments pin must be connected to ground or must be written with logic 0.

Common cathode

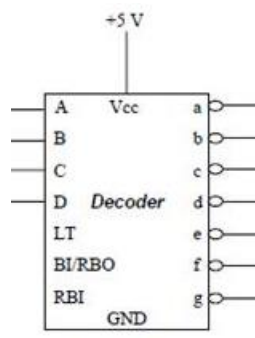
In this configuration Common pin is short with ground; the other open terminals of LEDs need to be provided with logic high signal to glow.

Driving 7-segment Display

7-segment display provides terminal associated with each display segment and can be controlled easily. There are two ways how we can control the display device.

- Using 7-segment decoder IC

7-segment decoder IC controls the ON/OFF condition of each display segment itself based on the input combination received at input pins. The 7-segment decoder simply decodes the input combination, and glows segments only required to get the desired number.

	Decimal or Function	Inputs							Outputs							Note
		LT	RBI	A3	A2	A1	A0	BI/RBO	a	b	c	d	e	f	g	
	0	H	H	L	L	L	L	H	L	L	L	L	L	L	H	(Note
	1	H	X	L	L	L	H	H	H	L	L	H	H	H	H	(Note
	2	H	X	L	L	H	L	H	L	L	H	L	L	H	L	
	3	H	X	L	L	H	H	H	L	L	L	L	H	H	L	
	4	H	X	L	H	L	L	H	H	L	L	H	H	L	L	
	5	H	X	L	H	L	H	H	L	H	L	L	H	L	L	
	6	H	X	L	H	H	L	H	H	H	L	L	L	L	L	
	7	H	X	L	H	H	H	H	L	L	L	H	H	H	H	
	8	H	X	H	L	L	L	H	L	L	L	L	L	L	L	
	9	H	X	H	L	L	H	H	L	L	L	H	H	L	L	
	10	H	X	H	L	H	L	H	H	H	H	L	L	H	L	
	11	H	X	H	L	H	H	H	H	H	L	L	H	H	L	
	12	H	X	H	H	L	L	H	H	L	H	H	H	L	L	
	13	H	X	H	H	L	H	H	L	H	H	L	H	L	L	
14	H	X	H	H	H	L	H	H	H	H	L	L	L	L		
15	H	X	H	H	H	H	H	H	H	H	H	H	H	H		
	$\overline{\text{BI}}$	X	X	X	X	X	X	L	H	H	H	H	H	H	H	(Note
	$\overline{\text{RBI}}$	H	L	L	L	L	L	L	H	H	H	H	H	H	H	(Note
	$\overline{\text{LT}}$	L	X	X	X	X	X	H	L	L	L	L	L	L	L	(Note
7-segment decoder(74LS47)	Truth Table For Driving 7 segment through 74LS47															

Testing 7 segment with Lamp Test

Lamp test is used to test 7 segment display, for this purpose on decoder IC LT pin is provided. LT is an active low input and when a 0 is written to this pin provided BI.RBO is kept High, all display segments glow with active low output.

- Implementing decoder algorithm in Microcontroller

To drive 7-segments with Microcontroller, algorithm is implemented to control each segment unit individually.

Program

- Using 7-segment decoder IC

```
#include<avr/io.h>
#include<util/delay.h>
int main(void)
{
    DDRA=0b11111111;
    char y;
    while(1)
    {
        y=0x03;//to display Digit 3, BCD value of 3 should be input to
        //ABCD
        PORTA=0b11111100 // (PA6)BI=1, (PA5)RBI=1, (PA4)LT=1, (PA3PA2PA1PA0)
        //ABCD=1100 (PA7 not connected)
```

```
}}
```

- Implementing decoder algorithm in Microcontroller

```
#include<avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>
int main(void)
{
    DDRA=0b11111111;
    char y;
    while(1)
    {
        y=0x03; // to display Digit 3, corresponding values of abcdefg.is
                //assigned to port A 'a' being LSB of portA
        PORTA= 0b10110000; // abcd and g are ON so they are kept as '0' all else
        //are kept '1'
    }
}
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a)
 - i. Driving 7-segment through Decoder IC

To drive 7-segment through decoder IC connect PORTA with 7-segment decoder IC header with PINA6 connected to BI', PINA5 connected to RBI and PINA4 connected to LT' of decoder IC

ii. Driving 7-segment through Microcontroller

To test decoding algorithm implemented in Microcontroller connect PORTA with 7-segment header with pin 'a' at header connected to LSB.

b) Take the USB cable and connect the USB connector to the development board USB B type connector, connect the other end of cable to the PC USB port.

c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.

10. Launch Khazama programming software and select ATmega162 from the drop down list.

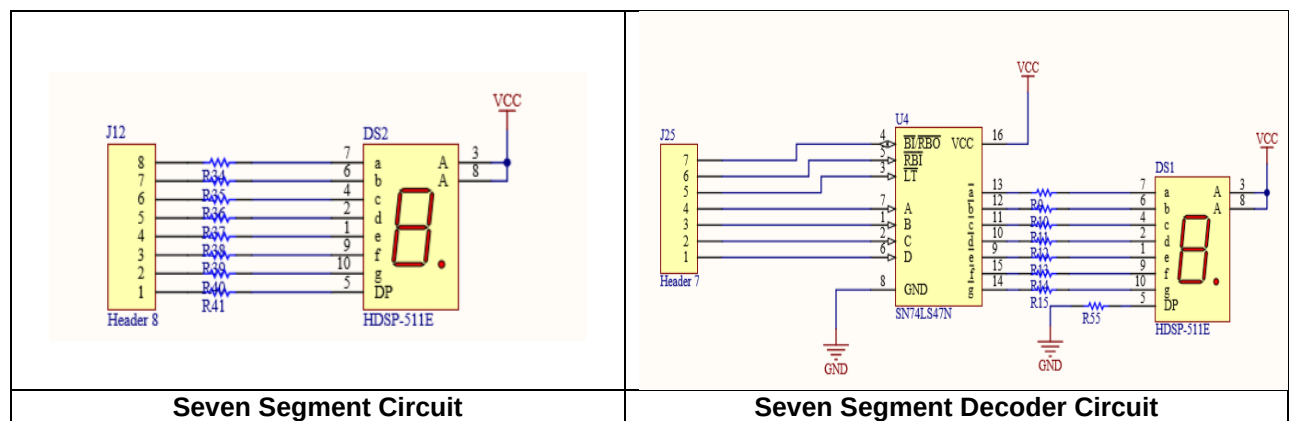
11. Click on  button and locate the HEX file generated from IDE and click on "Auto Program" to dump your code on Microcontroller flash memory.

12. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

Observation

Program your Microcontroller to display a numeric digit on 7-segment and observe the display. Program the microcontroller to implement both methods of driving 7-segment.

Circuit diagram



Exercise

1. What are the two types of 7-segment display? Relate these types with active high and active low enable segments.

[illegible]

2. Write down a look up table to display A, b, C, d, E, and F on seven segment.

DIGIT	a	b	c	d	e	f	g	Dp
A								
b								
C								
d								
E								
F								

- Design a display stream on 7-segments, for decimal numbers using decoder IC. Make it visible using delays.

[illegible]

4. Design a hex-decimal display stream implementing look up in microcontroller.

[illegible]

Lab Session 06

Practice using Timers and Counters in microcontroller

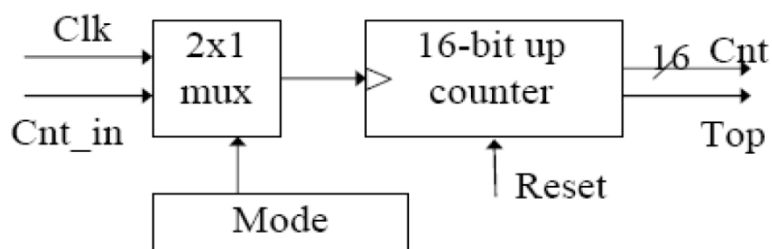
Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC

Timers and counters

Mostly Embedded system applications include different task which involve timing parameters to occur or some time measurements between the some events occurred. These tasks require high precision Timer/Counter units which exactly catch the instances. The difference of terminology has more to do with hoe we use the hardware unit. Timers are used when we need to get the time elapsed between two events while counters simply count some external events.

The following diagram shows how the same hardware unit is used for Timers/Counters modes.



Clock and Pre-scaling:

Timers/Counters operate on a clock signal, up counters usually reset to 0 and increment their value with single clock tick. An overflow occurs when their count reaches to maximum value that can be hold by the count register. For an indication of overflow they set some interrupt flag to report Microcontroller. For counters the clock signal is replaced by external even needs to count.

Resolution:

The least time stamp a Timer can measure known as resolution depends on the clock signal operating Timer unit. It is simply the inverse of input clock signal time period.

With system clock frequency Timers provide highly précised timing measurements.

$$\text{Resolution} = 1/\text{clk_in};$$

Range:

The maximum Time duration a timer can measure is called *Range*. It depends on the size of the count register and the input clock frequency. For a given register size we can increase the range of the timer by slowing down the input clock using pre-scalar.

Pre-scalars are used to increase the range of Timer; they basically divide the input clock of timer by a certain unit.

$$\text{Range} = \text{Resolution} \times 2^n; n \text{ is the size of timer/ counter register.}$$

Without pre-scaling: $\text{Resolution} = 1/\text{clk_io}$

With pre-scaling: $\text{Resolution} = (1/\text{clk_io}) \times p$; p is the pr-scale factor

ATmega16 Timers/Counters:

In ATmega16 there are two 8-bit (Timer 0 and 2) and one 16-bit timer/counter (Timer1), each with separate compare modes. These timers and counters offer different modes of operation with compare modes in which the hardware itself compares timer/counter register value with user defined value placed in **OCR** register and performs operation on GPIO as per configuration of timer/counter.

Following registers are associated with timer/counter unit.

TCNT: Timer/Counter register holds count value of timer/counter. Value starts from BOTTOM to TOP. BOTTOM indicates the lowest value i.e. 0x00 and TOP can be either user defined or equal to MAX that is 0xFF for 8-bit counter/timer and 0xFFFF for 16-bit timer/counter.

TCCR_x: Timer counter configuration register, holds necessary bits for configuring timer/counter clock source, desired modes of operation etc.

TCCR0 (Timer/Counter 0 configuration register)

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Clock source selection:

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$ /(No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Wave form generation mode: (for normal timer/counter application must be 00)

Mode	WGM01 (CTC0)	WGM00 (PWM0)	Timer/Counter Mode of Operation	TOP	Update of OCR0	TOV0 Flag Set-on
0	0	0	Normal	0xFF	Immediate	MAX
1	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	1	0	CTC	OCR0	Immediate	MAX
3	1	1	Fast PWM	0xFF	BOTTOM	MAX

Compare Output Match action: (for normal timer/counter application)

COM01	COM00	Description
0	0	Normal port operation, OC0 disconnected.
0	1	Toggle OC0 on compare match
1	0	Clear OC0 on compare match
1	1	Set OC0 on compare match

Other registers:

OCR0: Output compare register holds value for timer/counter register to be compared with.

TIMSK: used to enable timer/counter associated interrupts. A specific interrupt is enabled if the respective bit in TIMSK is set to 1 and interrupt bit (I) in status register is set to 1.

TOIE0 and OCIE0 enable timer overflow and output compare interrupts respectively for timer0.

TIFR: Timer Interrupt flag register holds interrupt and flag bits associated with timers/counters.

OCF0 and TOV0 indicate successful comparison and overflow occurrence if set to 1.

Program

```
#include<avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>
int main(void)
{
    DDRA |= (1 << 0);          // Set LED as output
    TCCR1B |= (1 << CS10)|(1<<CS12); // Set up timer
    while (1)
    {
        if ( TCNT1 >= 49)
        {
            PORTA ^= (1 << 0);    // Toggle the LED
            TCNT1 = 0;            // Reset timer value
        }
    }
}
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a) Connect LSB of Port A PA0 to one LED in the LED bank say to D8.
 - b) Take the USB cable and connect the USB connector to the development board USB B type connector, connect the other end of cable to the PC USB port.

- c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.
7. Launch Khazama programming software and select ATmega162 from the drop down list.
8. Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.
9. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

Observation

Set the timer such that an LED blinks at 20Hz.

Exercise

1. Toggle an LED with a frequency of less than 50Hz.

This image shows a full page of blank, lined paper. It features approximately 20 evenly spaced horizontal blue or grey lines across its entire width, typical of notebook paper. There are no margins, text, or other markings on the page.

2. Create a user-defined pre-scale value to insert a larger delay, say of 1 minute, to toggle the LED.

[illegible]

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 07

Practice LCD interfacing with microcontroller

Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC
- LCD controller HD44780

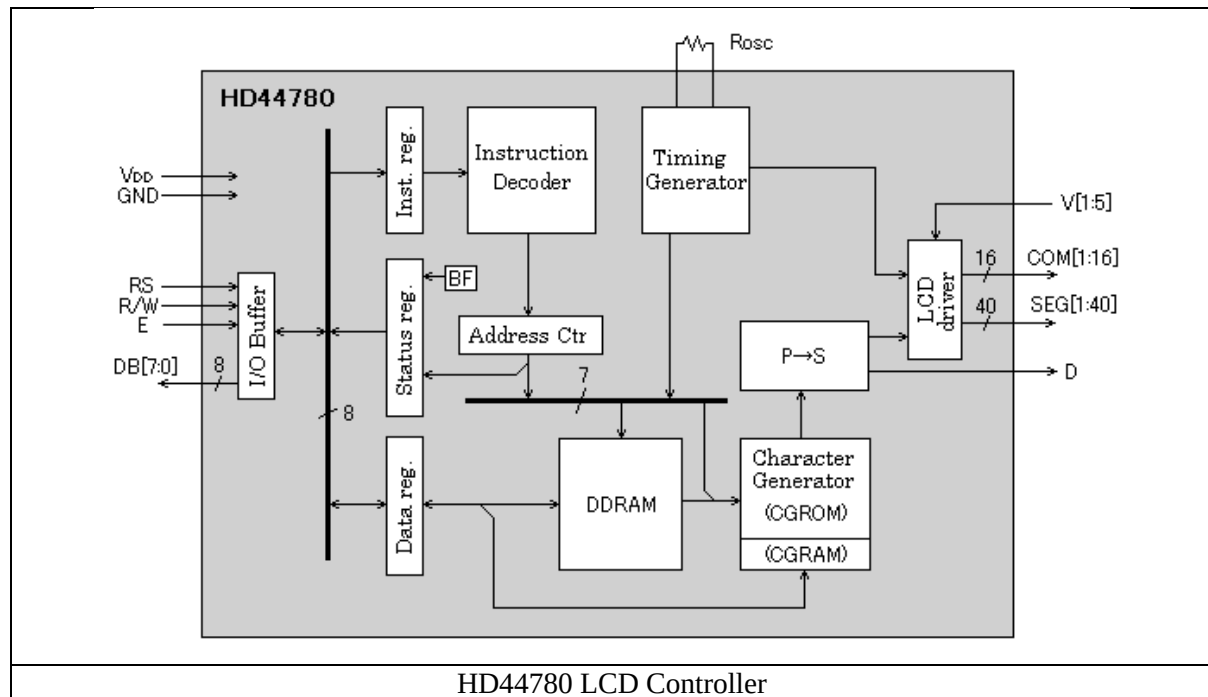
Introduction

- Liquid Crystal Display (LCD) is a low cost display device most common in embedded systems, capable to display text and images
- LCD provides user interface to observe the behavior of system, intermediate and final results of different operations, debug system flow by displaying necessary information.
- HD44780 is a monochrome, 5x8 and 5x10 dot matrix LCD controller that controls the instructions and data received from Microcontroller. LCD controller bears control and data lines to communicate with microcontroller.



- **Control Lines Functions:**
- RS(register select)
 - MCU Selects registers.
 - 0: Instruction register (for write)
 - 1: Data register (for write and read)
- R/W(read or write)

- MPU Selects read or write.
- 0: Write
- 1: Read
- E(enable)
 - MCU Starts data read/write.



LCD controller hosts following modules:

Instructions register:

The IR stores instruction codes, such as display clear and cursor shift etc.

CGRAM:

In the character generator RAM, the user can rewrite character patterns by program. For 5×8 dots, eight character patterns can be written, and for 5×10 dots, four character patterns can be written

CGROM:

The character generator ROM generates 5×8 dot or 5×10 dot character patterns from 8-bit character codes.

DDRAM:

Display data RAM (DDRAM) stores display data represented in 8-bit character codes. Each location of DDRAM is mapped to individual display matrix on LCD, to get a display at a specific location just set the pointer to the corresponding DDRAM address. Below is the addressing of DDRAM for a 20×4 LCD.

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	50	51	52	53
14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27
54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	60	61	62	63	64	65	66	67

There is an instruction set associated with the controller providing unique commands to handle data and instructions received from microcontroller. These commands are described below.

Instruction	Code										Description	Execution Time (max) (when f_{op} or f_{osc} is 270 kHz)
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0		
Clear display	0	0	0	0	0	0	0	0	0	1	Clears entire display and sets DDRAM address 0 in address counter.	
Return home	0	0	0	0	0	0	0	0	1	—	Sets DDRAM address 0 in address counter. Also returns display from being shifted to original position. DDRAM contents remain unchanged.	1.52 ms
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction and specifies display shift. These operations are performed during data write and read.	37 μ s
Display on/off control	0	0	0	0	0	0	1	D	C	B	Sets entire display (D) on/off, cursor on/off (C), and blinking of cursor position character (B).	37 μ s
Cursor or display shift	0	0	0	0	0	1	S/C	R/L	—	—	Moves cursor and shifts display without changing DDRAM contents.	37 μ s
Function set	0	0	0	0	1	DL	N	F	—	—	Sets interface data length (DL), number of display lines (N), and character font (F).	37 μ s
Set CGRAM address	0	0	0	1	ACG	ACG	ACG	ACG	ACG	ACG	Sets CGRAM address. CGRAM data is sent and received after this setting.	37 μ s
Set DDRAM address	0	0	1	ADD	ADD	ADD	ADD	ADD	ADD	ADD	Sets DDRAM address. DDRAM data is sent and received after this setting.	37 μ s
Read busy flag & address	0	1	BF	AC	AC	AC	AC	AC	AC	AC	Reads busy flag (BF) indicating internal operation is being performed and reads address counter contents.	0 μ s

LCD Controller Instruction Set

An internal reset circuit automatically initializes the HD44780U when the power is turned on. The following instructions are executed during the initialization. The busy flag (BF) is kept in the busy state until the initialization ends (BF = 1). The busy state lasts for 10 ms after VCC rises to 4.5 V.

1. Display clear

Command word: 0x01

2. Function set:

- DL = 1; 8-bit interface data
- N = 0; 1-line display
- F = 0; 5 ´ 8 dot character font

Command word: 0x38

3. Display on/off control:

- D = 0; Display off
- C = 0; Cursor off
- B = 0; Blinking off

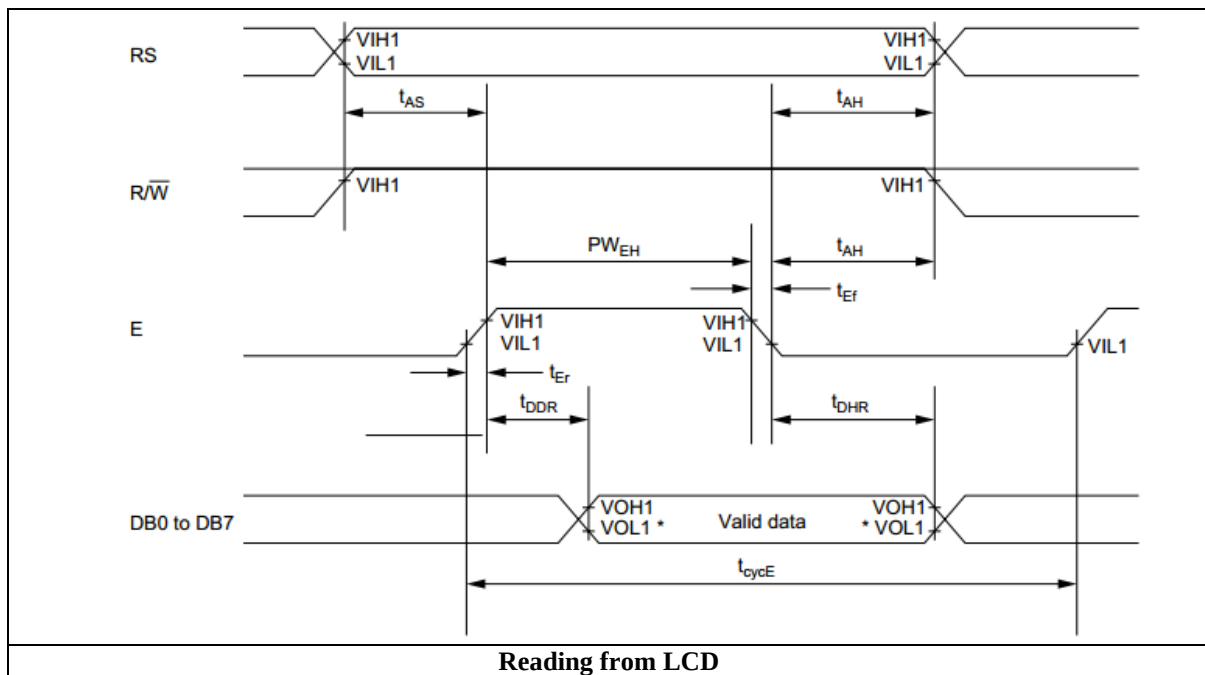
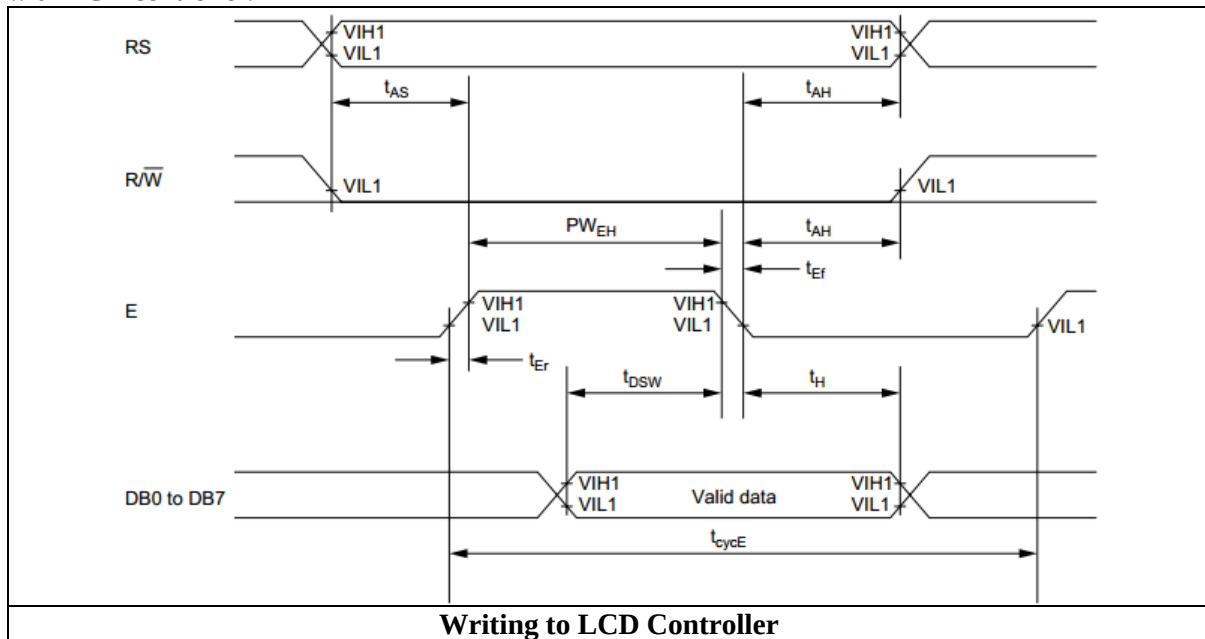
Command word: 0x0F

4. Entry mode set:

- I/D = 1; Increment by 1
- S = 0; No shift

Command word: 0x06

While interfacing with microcontroller following timings needs to be considered to be synchronized with LCD controller.



Program

```
#define F_CPU 100000UL

#include<avr/io.h>
#include<util/delay.h>

void _LCDClear(void);
void _LCDFunctSet(void);
void _LCD_DispCtrl(void);
void _LCDEntryMod(void);
void _PrintChar(char k);
```

```
int main(void)

{
    DDRA=0xFF;
    DDRE=0xFF;
    while(1)
    {
        _LDCDClear();
        _LCDFunctSet();
        _LCD_DispCtrl();
        _LCDEntryMod();
        char k='a';
        for(int i=0; i<5;i++)
        { _PrintChar(k);
          _delay_ms(500);
          k++;
        }
    }
    return 0;
}

void _LDCDClear(void)
{
    PORTA = 0x01;
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}

void _LCDFunctSet(void)
{
    PORTA = 0x3B; // or it can be 0x38
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}

void _LCD_DispCtrl(void)
{
    PORTA = 0x0F;
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}

void _LCDEntryMod(void)
{
    PORTA = 0x06;
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}

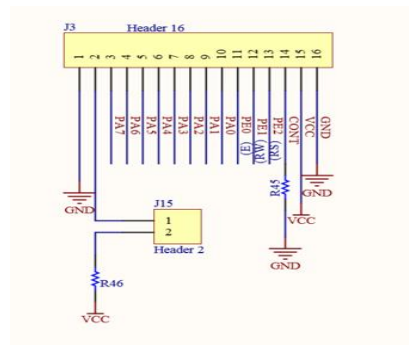
void _PrintChar(char K)
{
    PORTA = K;
```

```
    PORTE = 0x0D;  
    _delay_ms(5);  
    PORTE = 0x0E;  
    _delay_ms(5);  
    PORTE = 0x00;  
}
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a) Connect LCD module to the LCD header on top left side of the board (J3)..
 - b) Take the USB cable and connect the USB connector to the development board USB B type connector, connect the other end of cable to the PC USB port.
 - c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.
13. Launch Khazama programming software and select ATmega162 from the drop down list.
14. Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.
15. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

Circuit diagram



Observation

Program ATmega162 to initialize LCD module and write some characters on LCD after initialization and observe the display on LCD.

Exercise

1. Write a piece of code to initialize LCD module and display your name in the first line and semester in second one. Leave the remaining lines empty.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins or other markings on the paper.

2. Write Alphabet letters (only once) starting from location 0x44 of DDRAM.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 08

Practice keypad interfacing with microcontroller

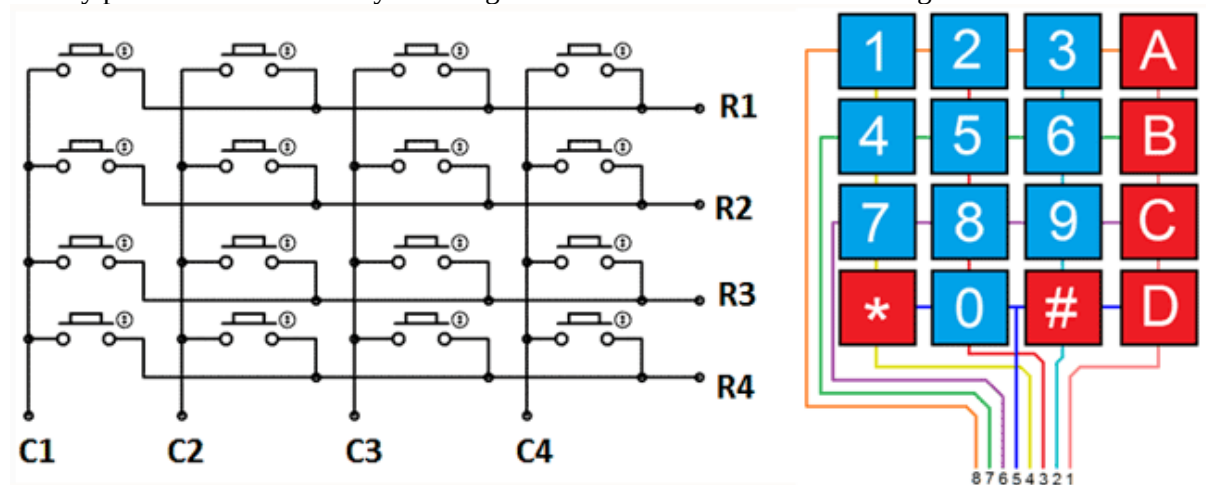
Requirements

- Microcontroller development kit along with necessary microcontroller ICs installed i.e AVR Microcontroller ATmega162
- 1- USB Power cable
- 16 jumper wires
- AVR Studio 7
- WinAVR- GNU GCC
- 4x4 Keypad

Interfacing keypad with microcontroller

Keypad is a set of buttons arranged in the form of a matrix. Keypads are common input devices used in embedded systems when there is a need of implementation of multiple buttons. Dedicating a separate GPIO to each button can lead to lack of GPIOs for other tasks when systems grow larger. Using keypad we can have more input buttons with quite less number of GPIOs used.

Matrix arrangement of grids provide unique intersection of Row(R) and Column(C) for each button, the key press can be detected by scanning the Rows and Columns in following manner.

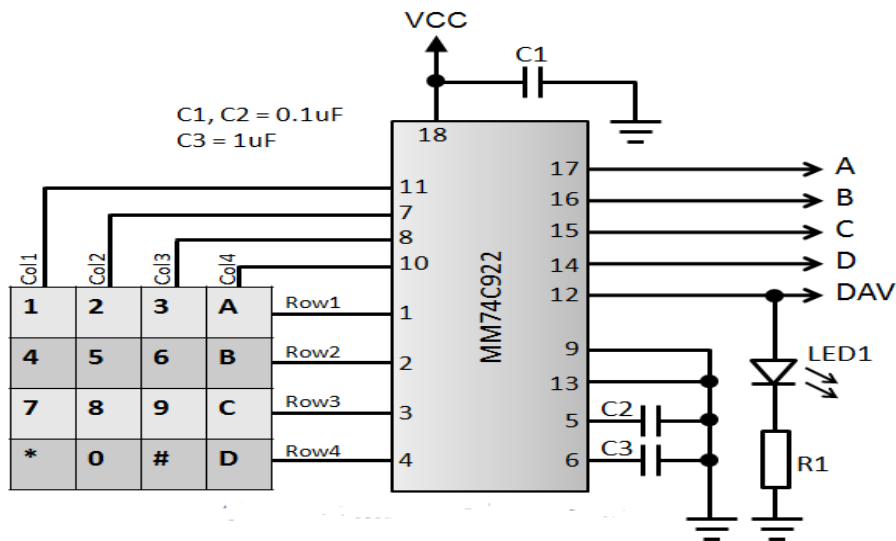


Decoding Mechanism

Rows and columns inside Keypad are arranged in a matrix combination such that each key (button) is mapped to a unique intersection point of Row and Columns. There is no electrical contact between Rows and Columns, when a key is pressed Row of the key goes short with corresponding Columns of the key pressed and a single short circuit sets up for the key pressed.

The MM74C922 CMOS key encoders provide all the necessary logic to fully encode an array of SPST switches. The keyboard scan can be implemented by either an external clock or external capacitor. These encoders also have on-chip pull-up devices which permit switches with up to 50 k Ω on resistance to be used. No diodes in the switch array are needed to eliminate ghost switches. The internal debounce circuit needs only a single external capacitor and can be defeated by omitting the capacitor. A Data Available output goes to a high level when a valid keyboard entry has been made. The Data Available

output returns to a low level when the entered key is released, even if another key is depressed. The Data Available will return high to indicate acceptance of the new key after a normal debounce period; this two-key roll-over is provided between any two switches. An internal register remembers the last key pressed even after the key is released



Program

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>

int GetKeyPressed(void);

int main(void)
{
    char
    digit[20]={'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D','\0'};
    DDRD=0xF0;
    DDRA=0xFF;
    DDRC=0x00;
    DDRB=(1<<PINB1);
    int key;
    PORTA=digit[16];
    while(1)
    {
        //TODO:: Please write your application code
        key= GetKeyPressed();
        if(key!=16)
        {
            PORTA=digit[key];
        }
    }
}

int GetKeyPressed(void)
```



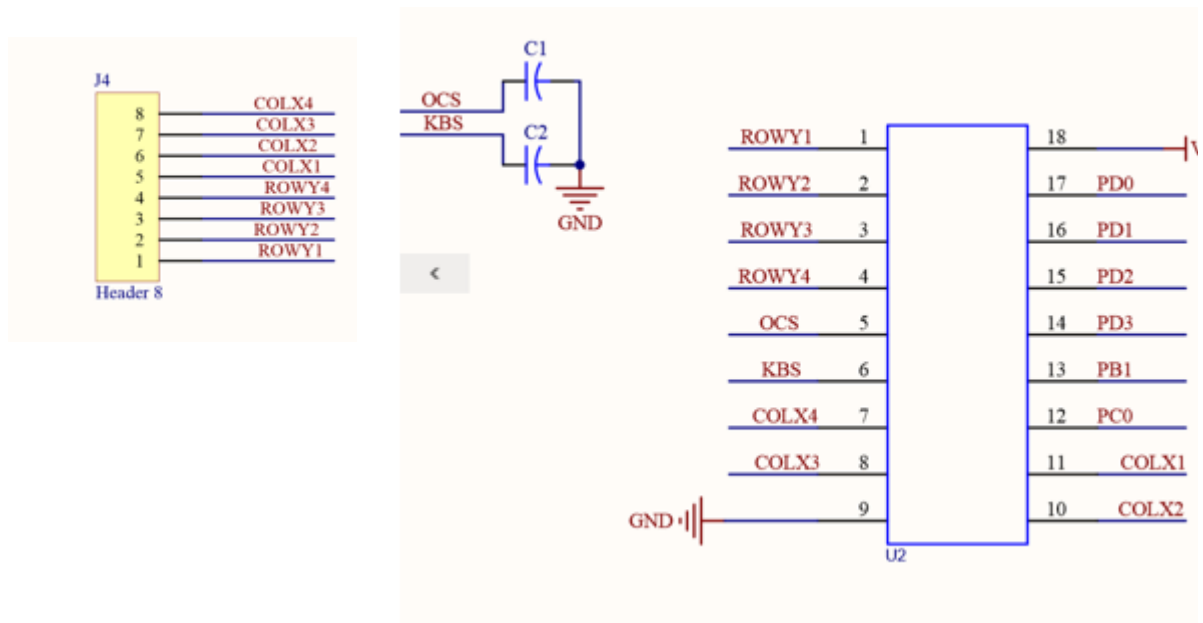
```
{  
    char x;  
    PORTB=0x00;  
    int data;  
    x=PINC;  
    if (x==0x01)  
    {  
        data=(PIND&0x0F);  
        return data;  
    }  
    return 16;  
}
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as follows:
 - a) Connect 4x4 Keypad to keypad header at bottom left of the board (J4). Also connect Port A to the LED bank such that PA0 is connected to the D8.
 - b) Take the USB cable and connect the USB connector to the development board USB B type connector, connect the other end of cable to the PC USB port.
 - c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.
7. Launch Khazama programming software and select ATmega162 from the drop down list.

8. Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.
9. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

Schematic diagram



Observation

Press different keys on the keypad and the hex values of the pressed key shall appear on the LED bank.

Exercise

1. Display the number pressed on the keypad on an LED bank.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 09

Practice ADC (Analog to Digital Convertor) interfacing with microcontroller

Requirements

- Microcontroller development kit along with necessary microcontroller i.e. AVR Microcontroller ATmega162 and 7-segment decoder ICs installed
- 1- USB Power cable
- 16 or more jumper wires
- ADC
- Capacitor (150pF)
- Resistors 10K Ω
- Function Generator
- Breadboard

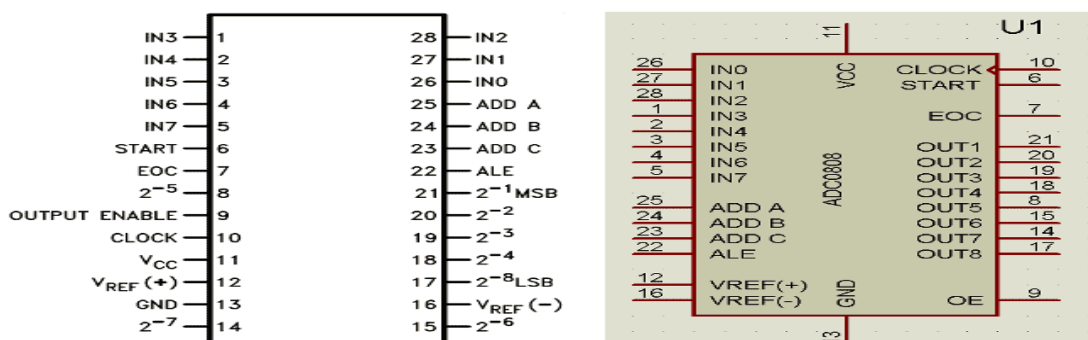
Analog to digital conversion

Analog-to-digital conversion is an electronic process in which a continuously variable (analog) signal is changed, without altering its essential content, into a multi-level (digital) signal. The input to an *analog-to-digital converter (ADC)* consists of a voltage that varies among a theoretically infinite number of values. Examples are sine waves, the waveforms representing human speech, and the signals from a conventional television camera. The output of the ADC, in contrast, has defined levels or states. The number of states is almost always a power of two -- that is, 2, 4, 8, 16, etc. The simplest digital signals have only two states, and are called binary. All whole numbers can be represented in binary form as strings of ones and zeros.

Application of ADC

Digital signals propagate more efficiently than analog signals, largely because digital impulses, which are well-defined and orderly, are easier for electronic circuits to distinguish from noise, which is chaotic. This is the chief advantage of digital modes in communications. Computers "talk" and "think" in terms of binary digital data; while a microprocessor can analyze analog data, it must be converted into digital form for the computer to make sense of it. A typical telephone modem makes use of an ADC to convert the incoming audio from a twisted-pair line into signals the computer can understand. In a digital signal processing system, an ADC is required if the signal input is analog.

Interfacing ADC with microcontroller



- IN0-IN7: Analog Input channels

- D0-D7: Data Lines
- A, B, C: Analog Channel select lines; A is LSB and C is MSB
- OE: Output enable signal
- ALE: Address Latch Enable
- EOC: End of Conversion signal
- Vref+/Vref-: Differential Reference voltage input
- Clock: External ADC clock input

ADC0808/0809 requires a clock pulse for conversion. We can provide it by using oscillator or by using microcontroller. In this lab we have applied frequency by using microcontroller.

We can select any input channel by using the Address lines, like we can select the input line IN0 by keeping all three address lines (ADDA, ADDB and ADDC) Low. If we want to select input channel IN2 then we need to keep ADDA, ADDC low and ADDB high.

Make appropriate connections by connecting output of ADC to the input Port C of microcontroller. Input analog voltage to be converted is applied at the required input channel of ADC through a potentiometer. Clock is provided through microcontroller. Address line pins ADDC, ADDB, ADDA are connected at PE2,PE1,PE0.

ALE (Address latch enable), SC (Start conversion), EOC (End of conversion), OE (Output enable) and clock pins are connected at PB5,PB1, PB3, PB4 and PB0(OC0) respectively.

Port D of microcontroller is connected to LED bank to observe the converted value.

Program

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
void adc(void);
void read_adc(void);

int main(void)
{
    DDRC=0b00000000; //output of ADC,input to micro controller
    DDRE=0xFF; //PE2=ADDC, PE1=ADDB, PE0=ADDA
    DDRB=0b00110011; // PB5=ALE, PB1=Start, //PB2=clock, PB3=EOC, PB4=OE,
    PB0=OC0;
    DDRD=0xFF; // output from controller to LED bank
    TCCR0|=(1<<WGM01)|(1<<COM00)|(1<<CS00);
    //TCNT0=0;
    OCR0=2;
    //Initialization
    //char number=0;
    PORTB=0b00001000;

    while(1)
    {

        adc(); //TODO:: Please write your application code
    }
}
} // main
```

```
void adc(void)
{
    PORTE=0x00; // Line 0 selected
    read_adc();
} //adc

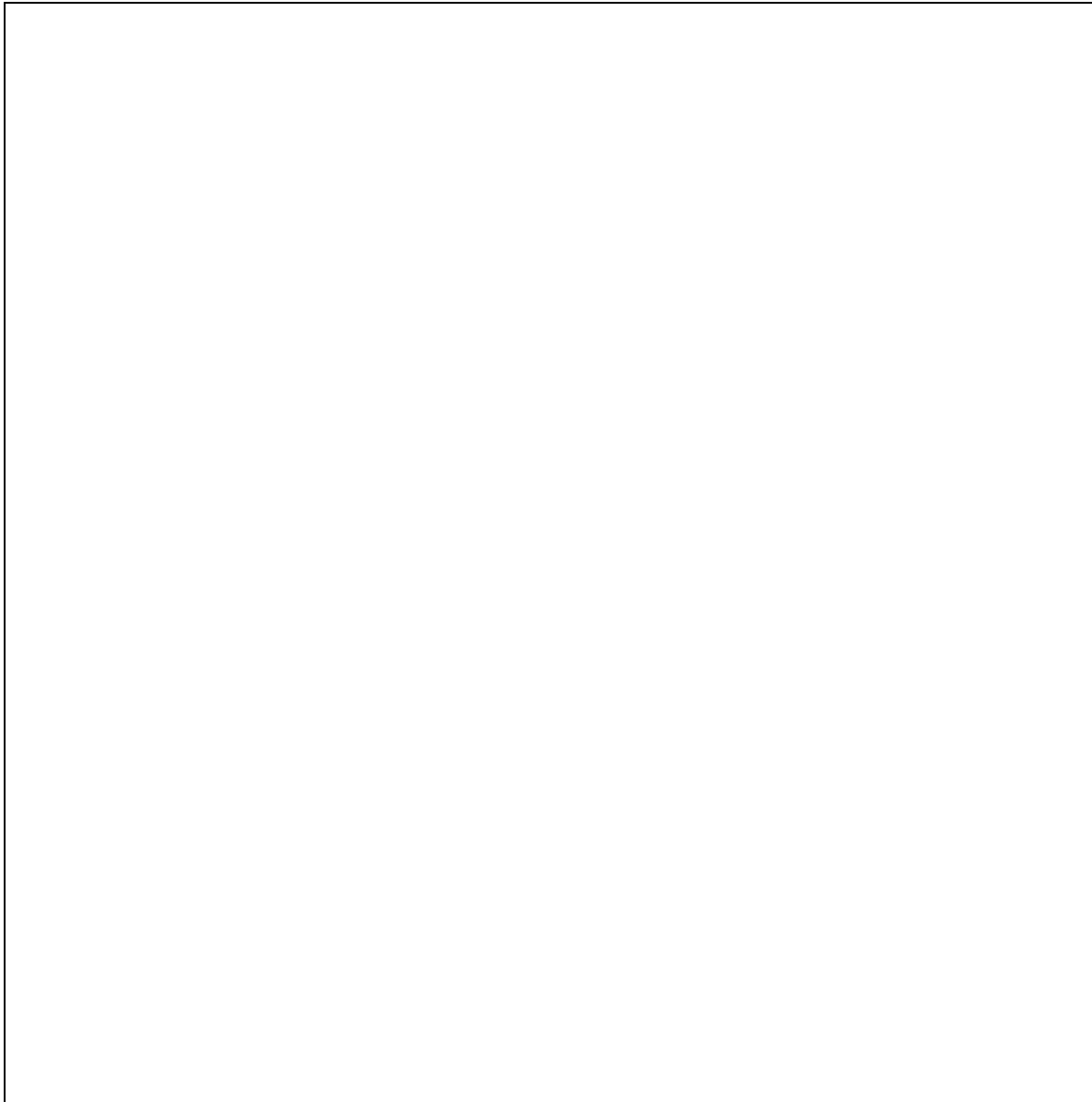
void read_adc(void)
{
    char num=0;
    PORTB=PORTB|0b00100010; //ALE and Start 1
    _delay_ms(100);
    PORTB=PORTB&0b11011101; // ALE and Start 0
    while((PINB&0b00001000)==0b00001000);
    while((PINB&0b00001000)==0b00000000);
    PORTB=PORTB|(0b00010000); //OE=1
    num=PINC;
    _delay_ms(200);
    PORTB=PORTB&(0b11101111); //OE=0
    PORTD=num;
}
```

Procedure

1. Create project in AVR Studio
 - a. Launch AVR Studio from Start menu.
 - b. Click Project and click on “NEW Project”
 - c. On next window select the AVR GCC Compiler
 - d. Enter your Project Name
 - e. Set the Location for your project
 - f. Press next to navigate to next window
 - g. Select the Debugger platform “AVR Simulator”
 - h. Select the target device “ATmega162”
 - i. Press Finish
2. Write C-Code in AVR studio
3. Compile the C-Code for error and warning removal
4. Simulate if required
5. Create Hex File
6. Make the AVR board connections as described:
 - a) Also connect Port D to the LED bank such that PD0 is connected to the D8. Connect the data out pins of ADC0808 to Port C. Make appropriate connections for the ADC with resistors and capacitor on breadboard.
 - b) Take the USB cable and connect the USB connector to the development board USB B type connector, connect the other end of cable to the PC USB port.
 - c) Make sure the jumper on the SPI programmer at the bottom of the board is in place.
7. Launch Khazama programming software and select ATmega162 from the drop down list.
8. Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.
9. Now the development board is fully configured and ready for experiment. Remove the SPI programmer Jumper.

Circuit diagram

Draw the circuit components interconnection.

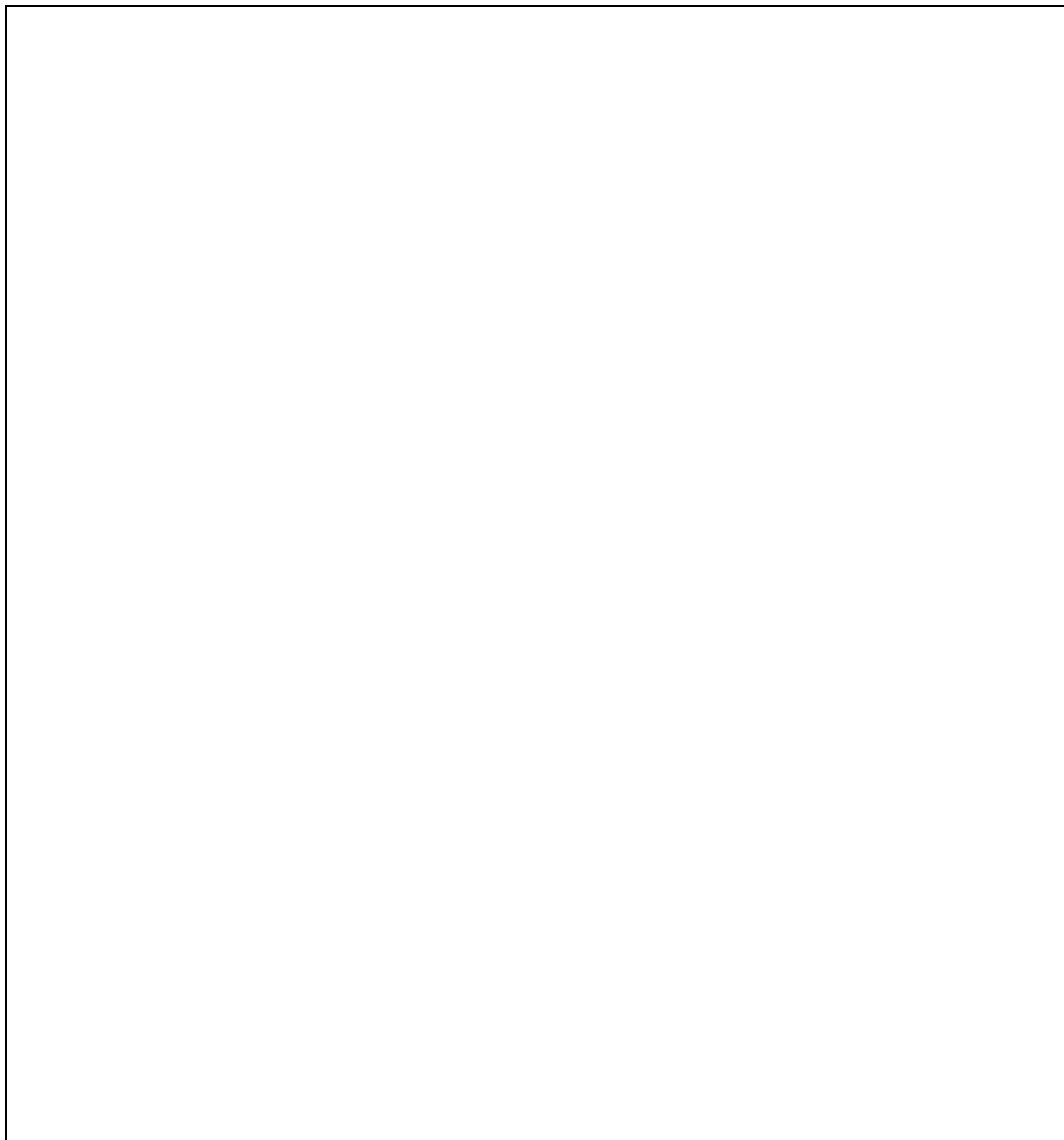


Observation

Interface ADC0808 with a Port of AVR board and observe the output on the LED .Briefly discuss your observations

Exercise

1. Practice analog to digital conversion with different input voltages and verify your results.



2. Write a piece of code to convert analog value obtained from wheel POT to digital value. When the received voltage is in the range of 2-4 volts glow all LEDs else keep them OFF.

[illegible]

3. Study the internal ADC feature of Atmega16 and report your findings.

This image shows a full page of blank, lined paper. It features approximately 28 horizontal blue lines spaced evenly apart, typical of standard notebook paper. The lines extend across the entire width of the page, leaving small margins at the top and bottom. There are no vertical lines, text, or other markings present.

Lab Session 10

Explore Serial Communication and AVR microcontroller interfacing with devices through RS-232(UART)

Serial communication is beneficial when we need to communicate bytes of data between devices that are separated by long distances and when device have few pins available for communication. Data transfer takes place bit by bit on single line of transmission; receiver continuously samples data with a clock frequency dictated by the transfer rate of communication. Serial communication can be categorized as synchronous or asynchronous serial communication.

UART

Universal asynchronous receiver/transmitter is the simplest serial protocol in which the two devices can communicate over three wire connection i.e. Rx (receive pin), Tx (transmit pin) and GND. The two devices get synchronized with each other by configuring data transmission rate and some indication signals.

UART configuration:

Configuration of the devices for communication over rs232 includes

- i- Baud rate configuration
Baud rate is the rate of signal change over the transmission line (usually higher than bit rate).
- ii- Number of bits transferred in single packet
While configuring rs232 it is needed to synchronize number of bits to be transferred in one packet. The number of bits in one packet can be 5, 6, 7 or 8 bits.
- iii- Stop bit
The data packet is followed by 1 or 2 stop bits as per configuration.
- iv- Parity bit
Parity bit is optional and can be disabled. If it is enabled then you can choose even or odd parity option. In even parity if number of 1's in the data packet received is even the parity bit received is 0 and 1 other wise. This logic becomes invert for odd parity.

Data transmission

Once configured both participants are ready to transfer and receive data. Transmitter shifts out data from transmit buffer after indicating a start of transmission by a high to low transition on Tx line. The data packet is followed by stop bit(s) and a parity bit, if enabled, for error detection.

ATMega16 UART and associated registers:

ATMega16 provides one serial channel and following registers are used in rs232 transmission. AtMega162 has two USART channels USART0 and USART1. Configuration registers are similar(Refer to the data sheet)

Configuration & Status Registers:

UCSRA- USART control & status register A

UCSRB- USART control & status register B

UCSRC- USART control & status register C

Bit	7	6	5	4	3	2	1	0	
	RXC	TXC	UDRE	FE	DOR	UPE	U2X	MPCM	UCSRA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

Bit 7 – RXC: USART Receive Complete

Bit 6 – TXC: USART Transmit Complete

- Bit 5 – UDRE: USART Data Register Empty
 Bit 4 – FE: Frame Error
 Bit 3 – DOR: Data OverRun
 Bit 2 – UPE: Parity Error
 Bit 1 – U2X: Double the USART Transmission Speed
 Bit 0 – MPCM: Multi-processor Communication Mode

Bit	7	6	5	4	3	2	1	0	
	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	UCSRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bit 7 – RXCIE: RX Complete Interrupt Enable
 Bit 6 – TXCIE: TX Complete Interrupt Enable
 Bit 5 – UDRIE: USART Data Register Empty Interrupt Enable
 Bit 4 – RXEN: Receiver Enable
 Bit 3 – TXEN: Transmitter Enable
 Bit 2 – UCSZ2: Character Size
 Bit 1 – RXB8: Receive Data Bit 8
 Bit 0 – TXB8: Transmit Data Bit 8

Bit	7	6	5	4	3	2	1	0	
	URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL	UCSRC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	0	0	0	0	1	1	0	

- Bit 7 – URSEL: Register Select
 Bit 6 – UMSEL: USART Mode Select
 Bit 5:4 – UPM1:0: Parity Mode
 Bit 3 – USBS: Stop Bit Select
 Bit 2:1 – UCSZ1:0: Character Size
 Bit 0 – UCPOL: Clock Polarity
 Bit configurations:

USBS	Stop Bit(s)
0	1-bit
1	2-bit

UMSEL	Mode
0	Asynchronous Operation
1	Synchronous Operation

UPM1	UPM0	Parity Mode
0	0	Disabled
0	1	Reserved
1	0	Enabled, Even Parity
1	1	Enabled, Odd Parity

Baud Rate Register:

Bit	15	14	13	12	11	10	9	8	
	URSEL	–	–	–	UBRR[11:8]				UBRRH
	UBRR[7:0]								UBRRL
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

With the value loaded in baud rate register baud rate of transmission is calculated as follows.

$$BAUD = \frac{f_{osc}}{16(UBRR + 1)}$$

If U2X bit in UCSRA is set to one for double seed,

$$BAUD = \frac{f_{osc}}{8(UBRR + 1)}$$

Bit 15- URSEL is Register select it is read as zero when reading UBRRH. The URSEL must be zero when writing the UBRRH.

USART Data Register:

Bit	7	6	5	4	3	2	1	0	
	RXB[7:0]								UDR (Read)
	TXB[7:0]								UDR (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

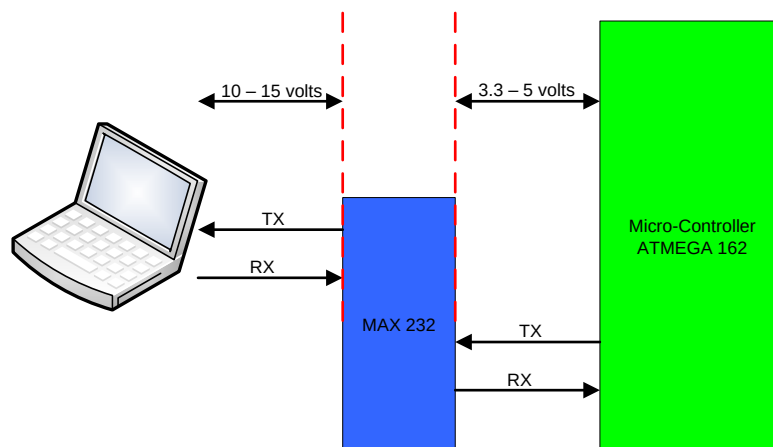
The transmit buffer can only be written when the UDRE Flag in the UCSRA Register is set.

When data is written to the transmit buffer, and the Transmitter is enabled, the Transmitter will load the data into the Transmit Shift Register when the Shift Register is empty. On the other side receiver continuously polls (Receive complete) RXC bit in UCSRA, when it is 1 it reads UDR value.

Interfacing AVR with PC:

Interfacing microcontroller with PC via USART is quite easy by cross connection of Rx and Tx but the data lines of PC are not similar to that of microcontroller in electrical characteristics. Microcontroller works on TTL voltage level that is 0V for Low logic and 5V for high logic whereas our computer's RS232 voltage level are +3 V to +15 V for low logic and -3 V to -15 V for high logic. Therefore we use a level shifter IC MAX232 that basically changes the TTL voltage level to RS232 and vice versa.

Block Diagram:



Exercise

1. Explore Baud rate. Give value loaded in the baud rate register to set the desired baud rate. Explore the usage of U2X bit in USCR register.

[illegible]

2. With 1 MHz input clock for 9600 Baud rate give the required baud value.

3. Design an algorithm to show steps of transmission and reception of data

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 11

Practice AVR microcontroller interfacing with devices through RS-232(UART)

Setup description for communication between two microcontrollers

Connect two microcontrollers where one microcontroller sends a data to the other microcontroller via USART interface. Connect Tx pin of transmitter AVR with Rx pin of receiver microcontroller. Connect a push button to PA0 of transmitter. If the transmitter microcontroller senses the push button as pressed it sends an 8 bit stream to the receiver microcontroller and it continues to send the same stream with a delay as long as push button is pressed. At the receiver, as soon as the transmission is complete, it is checked if the received data is same stream as was sent and if so it toggles the LED at PA0.

Program

Transmitter Code:

```
TX pin = transmit #include <avr/io.h>
RX pin = receive #define F_CPU 1000000UL
#include<util/delay.h>

int main(void)
{
    DDRD|=1<<PD1; pd1 as tx making it output
    DDRD&=~(1<<PD0); pd0 as rx pin means to receive or input
    PORTD|=1<<PD0; enable pull up resitor on rx pin
    DDRA=0x00; port a becomes input
    //int UBBRValue=25;
    UBRR0H=0x00;
    UBRR0L=0x05;
    UCSRB=(1<<TXEN0); transmit enable
    UCSRC=(1<<USBS0)|(3<<UCSZ00);
    char y=0;
    while(1)
    {
        y=PINA&0b00000001; read PA0 only
        if (y==0) if switch button pressed
        {
            while(!(UCSR0A&(1<<UDRE0))); wait till buffer is empty

            UDR0=0b11110000; UDR0 data registe, data loaded into it

            _delay_ms(1000);
        }
    }
}
```

Receiver Code:

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>

int main(void)
{
    DDRD&=~(1<<PD0); pd0 Rx pin input
    DDRD|=1<<PD1; pd1 Tx pin output
    DDRA=0xFF; port a output
```

```

DDRC=0xFF; port c output
//int UBRR_Value=25;
//UBRR0H=(unsigned char) (UBRR_Value>>8);
//UBRR0L=(unsigned char)UBRR_Value;
UBRR0H=0x00; same baud rate set
UBRR0L=0x05;
UCSR0B=(1<<RXEN0); receivr enable
UCSR0C=(1<<USBS0)|(3<<UCSZ00); same data frame set
unsigned char receiveData;

while(1)
{ PORTC=0;
  receiveData=0;
  while(!(UCSR0A&(1<<RXC0))); rxc0 means receive complete , hnece wait till byte receied
  receiveData=UDR0; udr0 is filled with data by the transmitter
  PORTC=receiveData;
  if (receiveData==0b11110000)
  {
    PORTA^=(1<<PA0); if rec dat matches, glow the led
    _delay_ms(1000);
  }
}
}

```

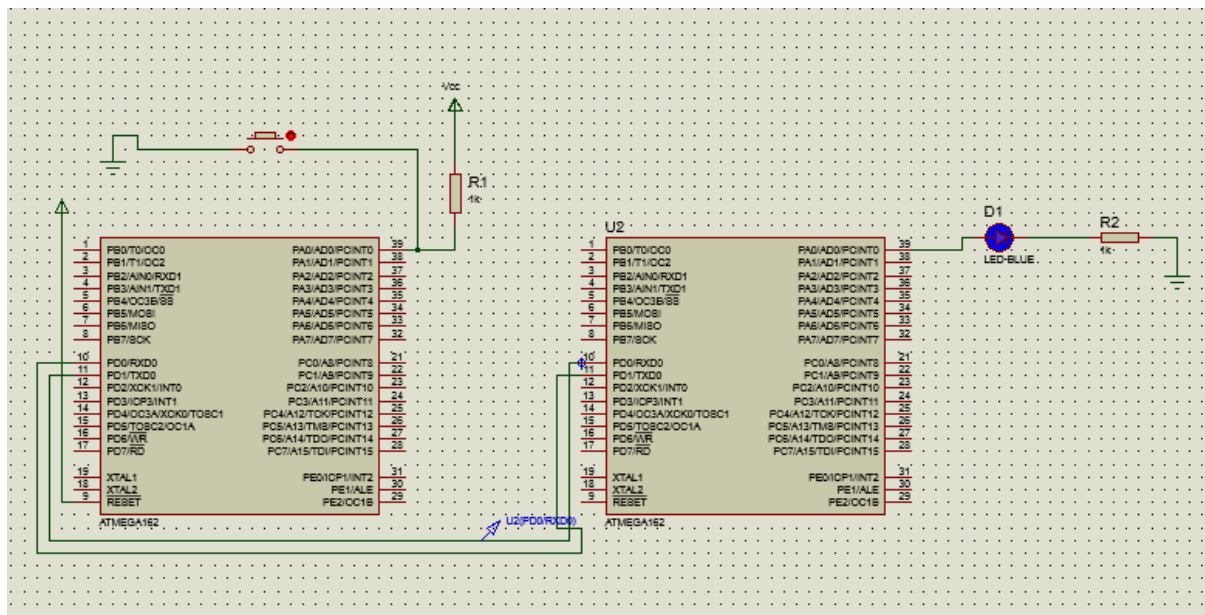
Take the USB cable and connect the USB connector PC USB port.

Launch Khazama programming software and select ATmega162 from the drop down list.

Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.

Now the development board is fully configured and ready for experiment.

Circuit diagram



Observation

Implement the above setup using two AVR kits and record your observations.

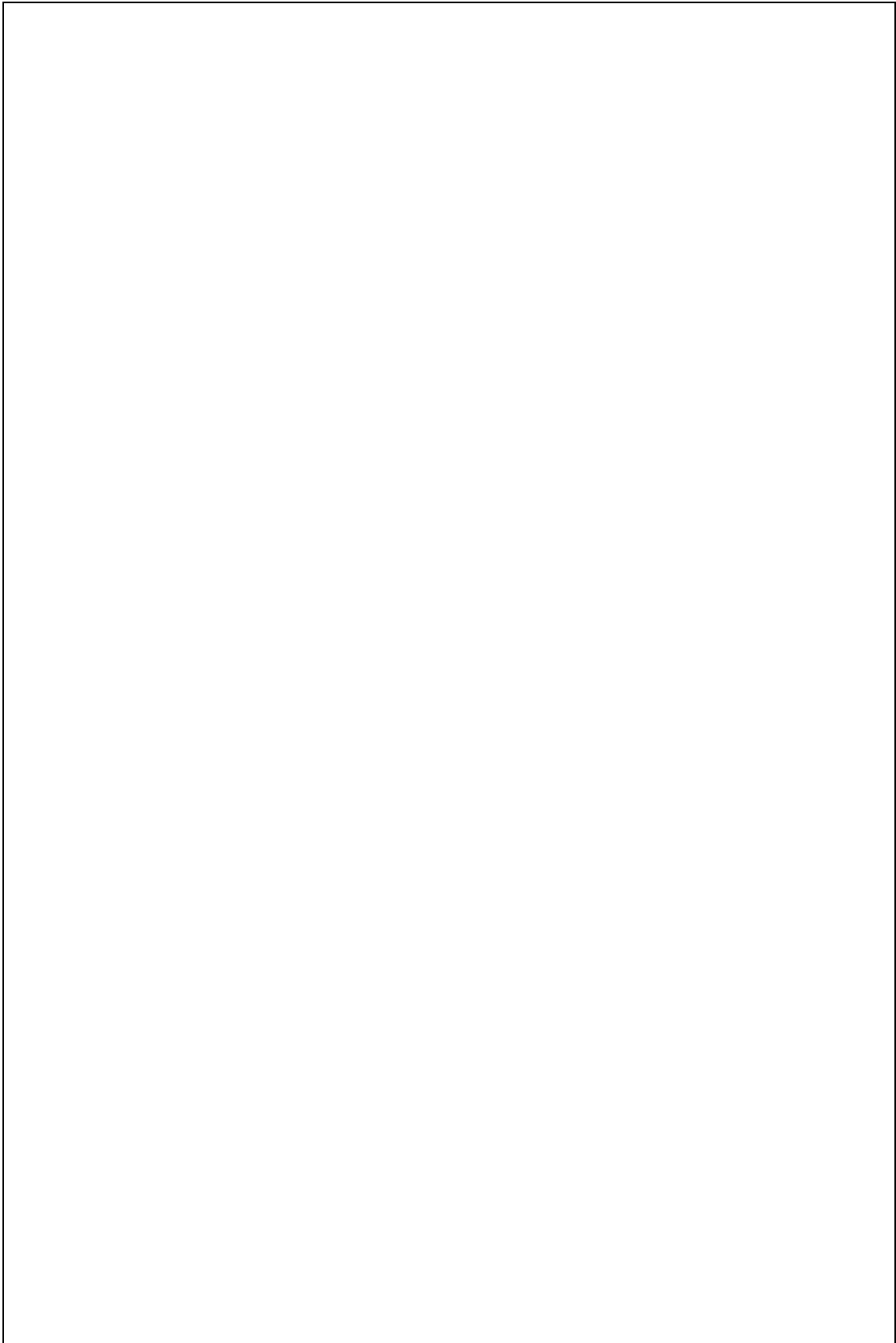
This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Exercise

1. Write a piece of code to send a stream over USART and receive the transmitted data and display on LEDs.

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are approximately 20 lines visible. The paper has a slight shadow on its right side, suggesting it's resting on a surface.

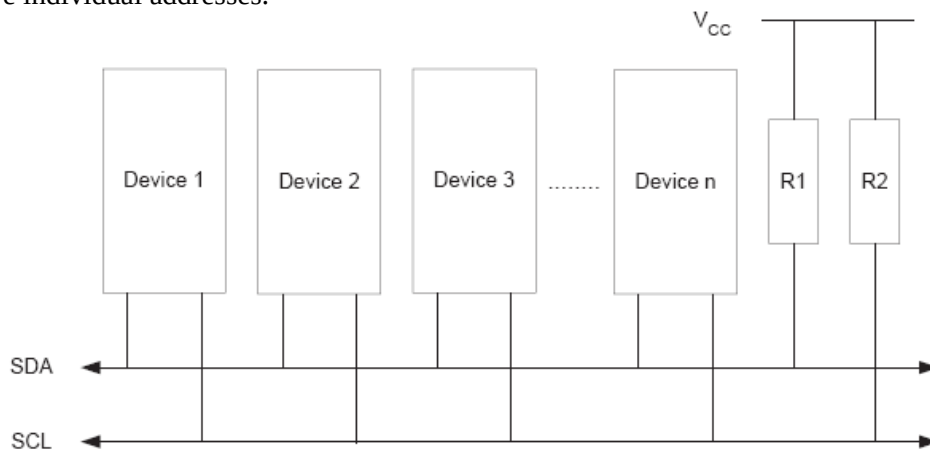
This image shows a single sheet of white paper with horizontal blue or grey ruling lines, typical of notebook paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



Lab Session 12

Practice microcontroller interfacing with devices through I²C Protocol

The Two-wire Serial Interface (TWI) is ideally suited for typical microcontroller applications. The TWI protocol allows the systems designer to interconnect up to 128 different devices using only two bi-directional bus lines, one for clock (SCL) and one for data (SDA). The only external hardware needed to implement the bus is a single pull-up resistor for each of the TWI bus lines. All devices connected to the bus have individual addresses.



I²C communication

I²C is synchronous serial communication protocol; in synchronous communication there exists a central clock line for synchronization of communication devices. It consists of Master Slave configuration; Master is normally responsible for maintaining synchronization with devices.

Master: The device that initiates and terminates a transmission. The Master also generates the SCL clock.

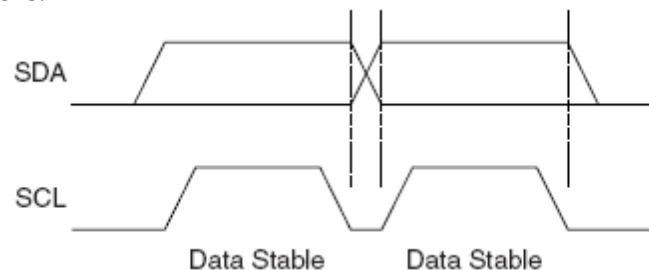
Slave: The device addressed by a Master.

Transmitter: The device placing data on the bus.

Receiver: The device reading data from the bus.

Data Stability:

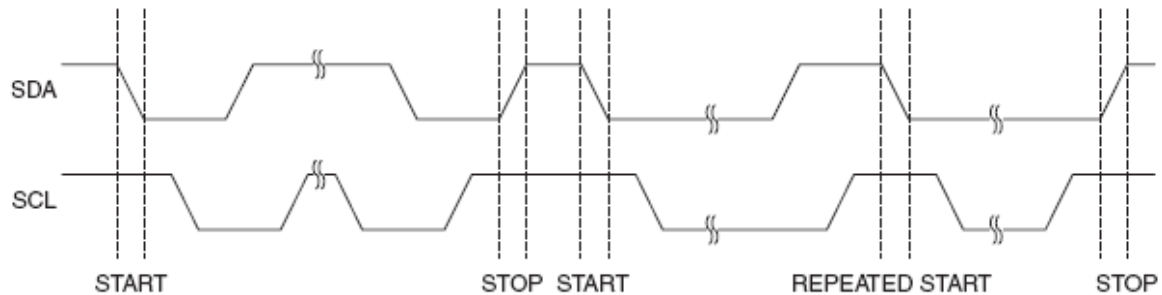
Each data bit transferred on the TWI bus is accompanied by a pulse on the clock line. The level of the data line must be stable when the clock line is high. The only exception to this rule is for generating start and stop conditions.



Start and Stop Condition:

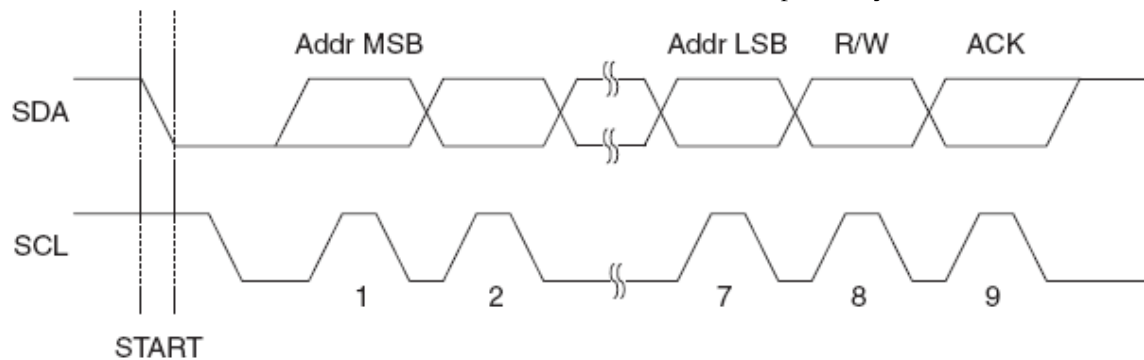
The Master initiates and terminates a data transmission. The transmission is initiated when the Master issues a START condition on the bus, and it is terminated when the Master issues a STOP condition. Between a START and a STOP condition, the bus is considered busy, and no other Master should try

to seize control of the bus. A special case occurs when a new START condition is issued between a START and STOP condition. This is referred to as a REPEATED START condition, and is used when the Master wishes to initiate a new transfer without releasing control of the bus. START and STOP conditions are signaled by changing the level of the SDA line when the SCL line is high.



Addressing format:

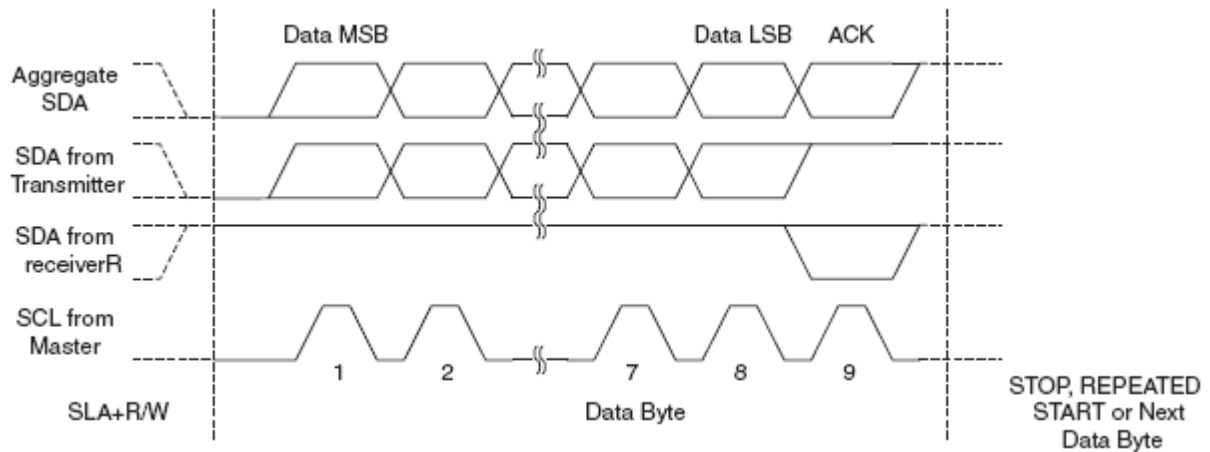
All address packets transmitted on the TWI bus are nine bits long, consisting of seven address bits, one READ/WRITE control bit and an acknowledge bit. If the READ/WRITE bit is set, a read operation is to be performed; otherwise a write operation should be performed. When a Slave recognizes that it is being addressed, it should acknowledge by pulling SDA low in the ninth SCL (ACK) cycle. If the addressed Slave is busy, or for some other reason cannot service the Master's request, the SDA line should be left high in the ACK clock cycle. The Master can then transmit a STOP condition, or a REPEATED START condition to initiate a new transmission. An address packet consisting of a Slave address and a READ or a WRITE bit is called SLA+R or SLA+W, respectively.



Each I2C has an associated hardwired address and few address bits are left open for programmable addressing. After connecting these pins to make complete address device is able to be addressed on the bus.

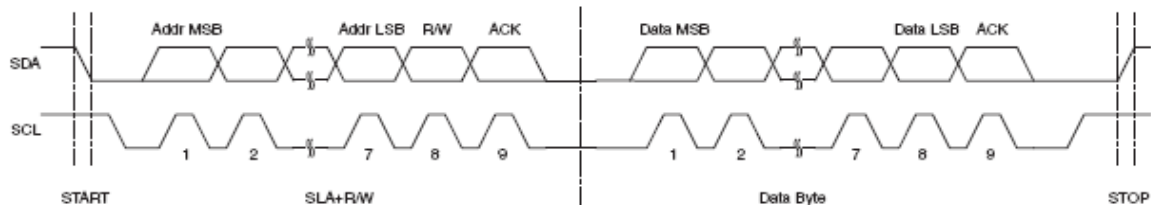
Data Format:

All data packets transmitted on the TWI bus are nine bits long, consisting of one data byte and an acknowledge bit. During a data transfer, the Master generates the clock and the START and STOP conditions, while the receiver is responsible for acknowledging the reception. An Acknowledge (ACK) is signaled by the receiver pulling the SDA line low during the ninth SCL cycle. If the receiver leaves the SDA line high, a NACK is signaled. When the receiver has received the last byte, or for some reason cannot receive any more bytes, it should inform the transmitter by sending a NACK after the final byte. The MSB of the data byte is transmitted first.



Complete Transmission Cycle:

A typical data transmission cycle of i2c consists of a START signal generated from master following the slave address packet with read/write signal (SLA+R/W). Each packet containing data or address is followed by ACK generated from the receiving device. After transmission of data master sends STOP signal.



ATMega16 I²C Module:

I²C module in ATMega16 is comprised of different components performing multiple necessary operations and interrupt related tasks some basic of them are listed below.

SDA and SCL: data and clock lines of I²C are mapped to GPIOs to interface I²C module with the rest of MCU system.

Bit rate generator unit: This unit controls the period of SCL when operating in a Master mode. The SCL period is controlled by settings in the TWI Bit Rate Register (TWBR) and the Prescaler bits in the TWI Status Register (TWSR). Slave operation does not depend on Bit Rate or Prescaler settings, but the CPU clock frequency in the Slave must be at least 16 times higher than the SCL frequency.

The SCL frequency is generated according to the following equation:

$$\text{SCL frequency} = \frac{\text{CPU Clock frequency}}{16 + 2(\text{TWBR}) \cdot 4^{\text{TWPS}}}$$

Where,

TWBR = Value of the TWI Bit Rate Register

TWPS = Value of the Prescaler bits in the TWI Status Register

Following registers are associated with I2C module for status and control configurations.

TWBR: TWI Bit Rate Generator

Bit	7	6	5	4	3	2	1	0	
	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The value in this register is used to calculate the SCL frequency to be generated by the master, according to the formula mentioned before.

TWCR: TWI Control Register

Bit	7	6	5	4	3	2	1	0	
	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE	TWCR
Read/Write	R/W	R/W	R/W	R/W	R	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TWSR: TWI Status Register

Bit	7	6	5	4	3	2	1	0	
	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

Bit rate Prescaler Selection:

TWPS1	TWPS0	Prescaler Value
0	0	1
0	1	4
1	0	16
1	1	64

TWDR: TWI Data Register

Bit	7	6	5	4	3	2	1	0	
	TWD7	TWD6	TWD5	TWD4	TWD3	TWD2	TWD1	TWD0	TWDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	1	1	1	1	

At the transmission side this data contains next byte to be transmitted while at receiving side it contains last byte of data received.

TWAR: TWI Address Register

Bit	7	6	5	4	3	2	1	0	
	TWA6	TWA5	TWA4	TWA3	TWA2	TWA1	TWA0	TWGCE	TWAR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	1	1	1	0	

The TWAR should be loaded with the 7-bit Slave address (in the seven most significant bits of TWAR) to which the TWI will respond when programmed as a Slave Transmitter or receiver. In multi-master systems, TWAR must be set in Masters which can be addressed as Slaves by other Masters.

Setup description

Connect SDA and SCL lines of Microcontroller with two separate pull up resistors of 4.7K connected to Vcc. Connect oscilloscope with SDA and SCL on separate channels.

Click on  button and locate the HEX file generated from IDE and click on “Auto Program” to dump your code on Microcontroller flash memory.

Now the development board is fully configured and ready for experiment.

Procedure and observation

Program your microcontroller to be the master of I²C communication and observe the clock line behavior.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Exercise

1. Interface PCF8574 Remote 8-Bit I/O Expander for I²C Bus with AVR Microcontroller as input port and as output port. Refer to the datasheet of Expander IC.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, providing a guide for writing. There are no margins, text, or other markings on the paper.

2. Draw the schematic diagram for the design in Exercise 1



Lab Session 13

Practice motor driver interfacing with microcontroller using Proteus

Motor driver

Motor driver L293d is a quadruple drive. This IC provides high bidirectional current of up to 1A at voltages from 4.5V to 36V. Bidirectional current is necessary for the DC motor to operate it in both the directions. L293d contains 2 drives so we can connect and operate 2 motors at a time. There are two control signals for each drive.

IN1 & IN2:

Using IN1 and IN2 we control the direction of the DC motor. For moving in one direction we have to provide high logic to 1 and low logic to other and for moving the motor in other direction we have to reverse the logic at both the pins with respect to the previous state. When we want to stop motor instantaneously we provide the same polarity at both the pins.

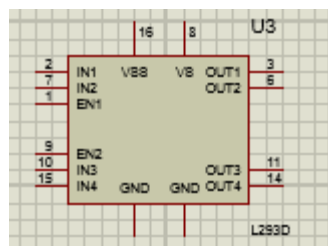
EN1 & EN2:

EN1 and EN2 enable and disable the drives. For enabling high logic is provided and for disabling low is provided. The table below best explains all the control conditions

EN	1A	2A	FUNCTION
H	L	H	Turn right
H	H	L	Turn left
H	L	L	Fast motor stop
H	H	H	Fast motor stop
L	X	X	Fast motor stop

L = low, H = high, X = don't care

Following is the pin configuration of L293D IC.



Program

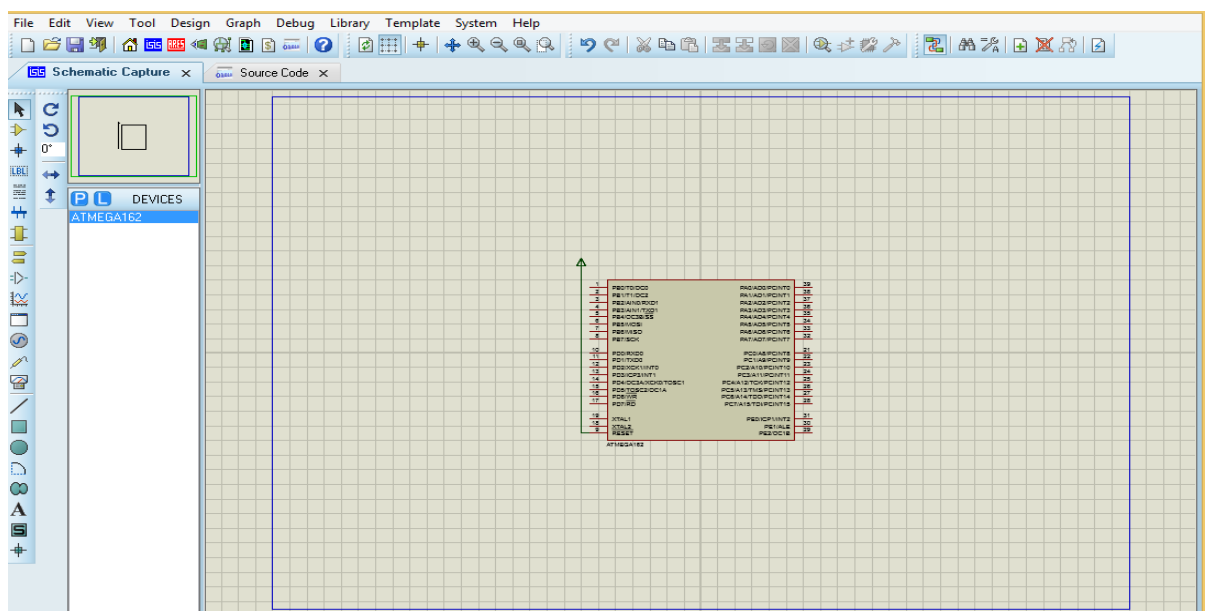
```
#include <avr/io.h>
#define F_CPU 1000000UL
#include<util/delay.h>

int main(void)
{
    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x00;
    char sw;

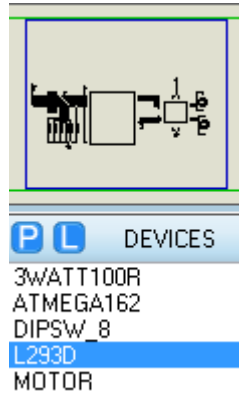
    while(1)
    {
        sw=PINB;
        PORTA=sw&(0b00000111);
        PORTC=sw&(0b00111000);
        _delay_ms(1000);
    }
}
```

Procedure

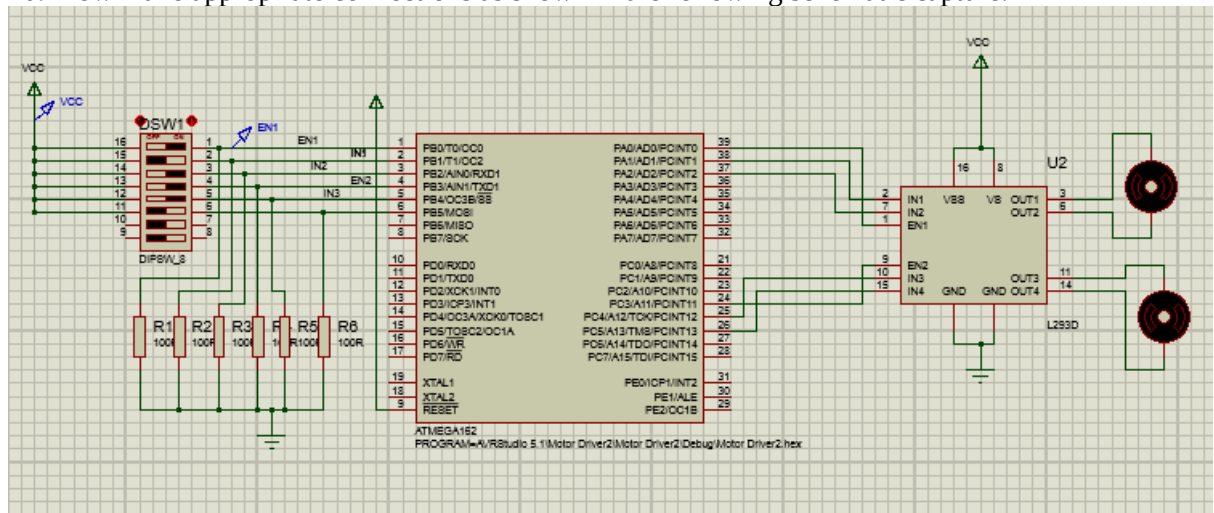
1. Launch Atmel Studio. Create new Project by selecting Atmega162 as device.
2. Write the given source code and compile to create hex file.
3. Now launch Proteus Design Suite select new project and write name of your project. Click next.
4. On next page, select create a schematic from selected template and select default. Click next.
5. Select create firmware project, choose AVR as family controller as Atmega162 and click finish.
6. From the menu bar select ISIS schematic capture. Schematic capture window will open with Atmega162 added.



7. From the leftmost pane select component mode, click on P Under Devices to pick all the components to be added on the schematic.
8. Add L293D, two simple DC Motors, 6 100 ohm resistors, DIPSW_8 on the workspace.



9. From the leftmost pane select terminals mode and select power and ground terminals. We need power supply of 5V for L293D IC and for the DIP Switches.
10. Now make appropriate connections as shown in the following schematic capture.

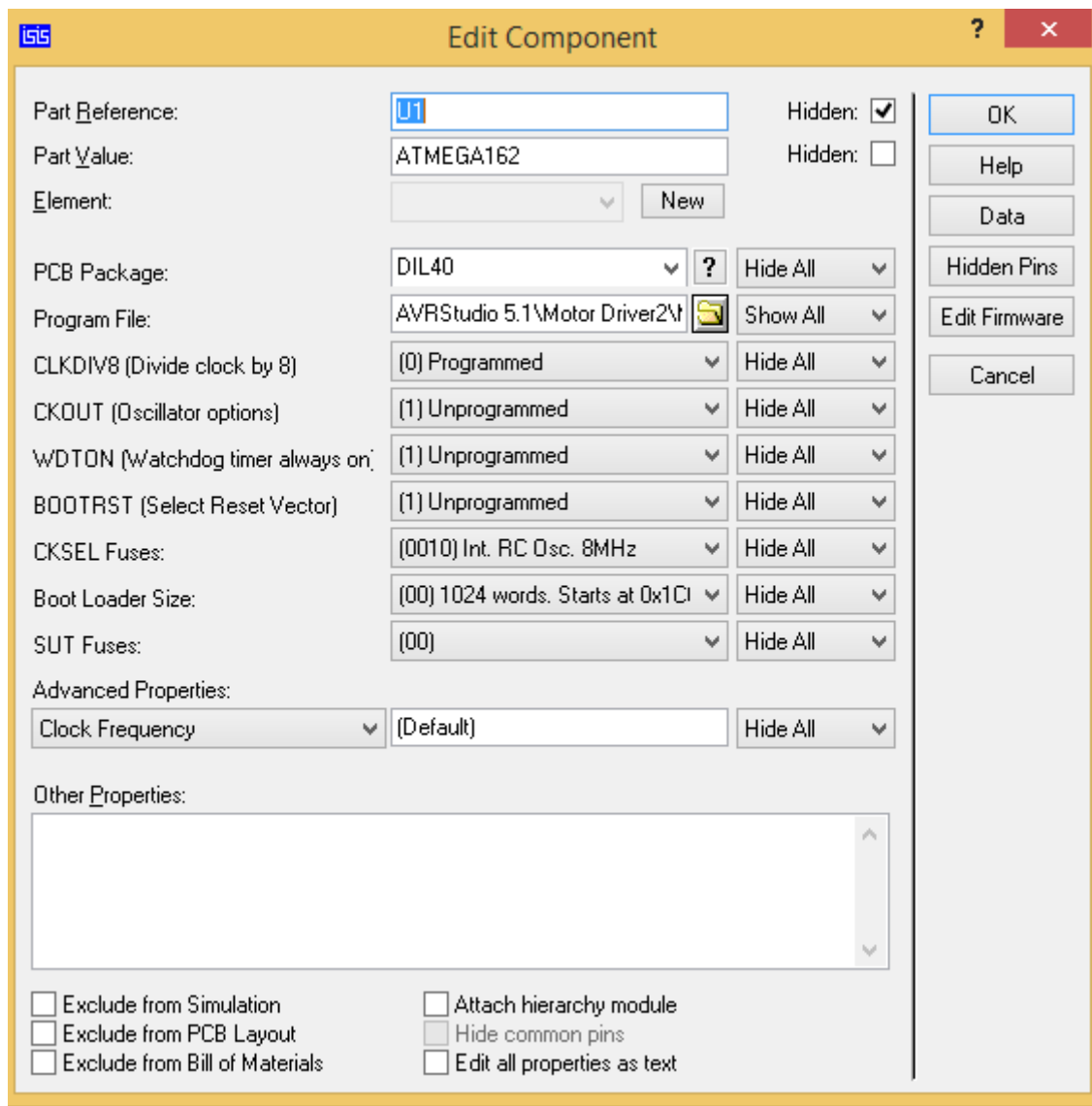


Here we have used PA0, PA1 and PA2 for EN1, IN1 and IN2 respectively. Similarly we have used output port PC3, PC4 and PC5 for EN2, IN3 and IN4 respectively.

On the input side we have used PORTB of AVR microcontroller and DIP switches are connected to give inputs.

PB0, PB1, PB2 are used as EN1, IN1 and IN2 respectively. PB3, PB4 and PB5 are used for EN2, IN3, and IN5 respectively.

11. Now double click on the microcontroller. An Edit Component dialogue box will open where make the following configurations.



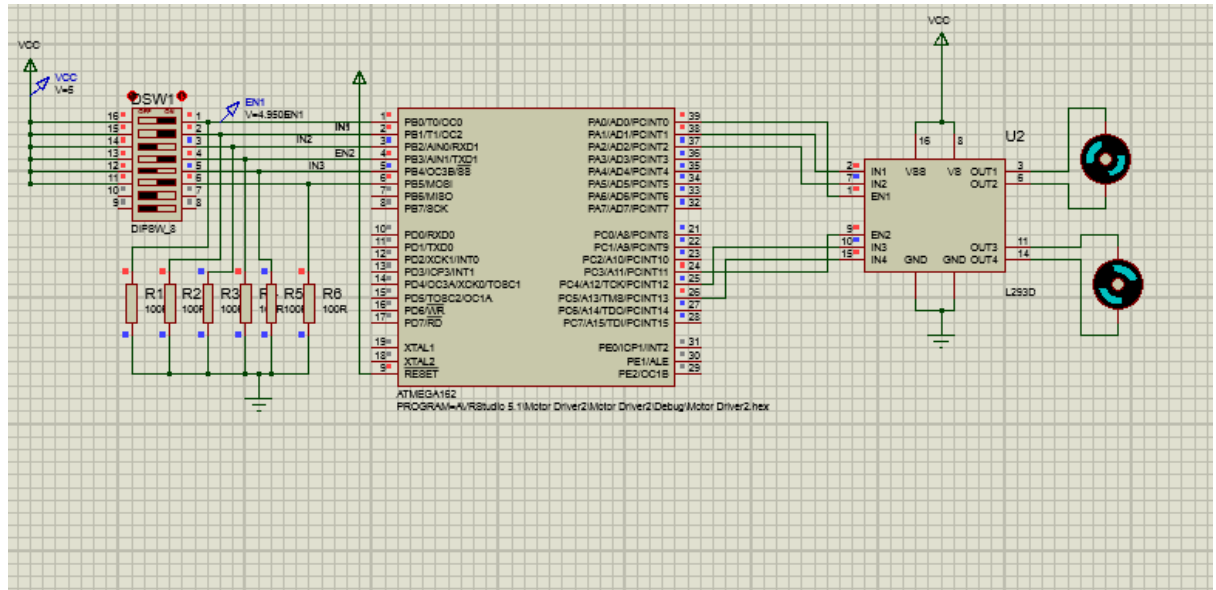
12. Browse in Program file to select the hex file created earlier and click ok.

13. Now run the simulation from the bottom bar by clicking on play button.



14. As you will run the simulation, the motors will run according to the inputs you give through dip switches.

15. You can rotate the motors clockwise or anticlockwise or you can stop them.



Observation

Briefly describe how the simulation works and turn the motors in different directions.

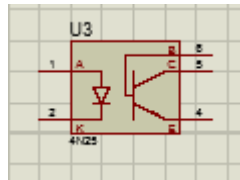
This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Lab Session 14

Practice isolating voltage levels between circuits through opto-isolator and microcontroller using Proteus

An opto-isolator is also called an opto-coupler. Opto-coupler is a component that transfer electrical signal between two isolated circuits with the help of light. An opto-coupler contains a source of light that transmits input signal and on the other end there is a photo sensor that senses the light and switch on the output device connected to it. In this way, two electrical signals, operating at different voltage levels, can coordinate with each other maintaining the isolation between them.

We need to use these sort of isolator where we do not want any interruption from the output device. Like for driving DC motor from Microcontroller we prefer to use this opto-coupler in order to prevent the noise signal from the motor that eventually affects the performance of the microcontroller if it is not kept isolated from the motor.



Here Pin A and K represent the source side and pin B C and E represent the load side. At the source there is an LED and at the load side there is a photo-transistor. When a signal is passed to the signal at the source side, LED emits radiation and on light reception the transistor turns on. Hence control signal from controller gets converted to light for triggering the light sensitive load driver.

Program

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>

int main(void)
{
    DDRD=0x00;
    DDRB=0xFF;
    char y;
    PORTB=0;
    while(1)
    {
        y=PIND;
        if(y==1)
        {
            PORTB^=(1<<0);
            _delay_ms(1000);
        }
    }
}
```

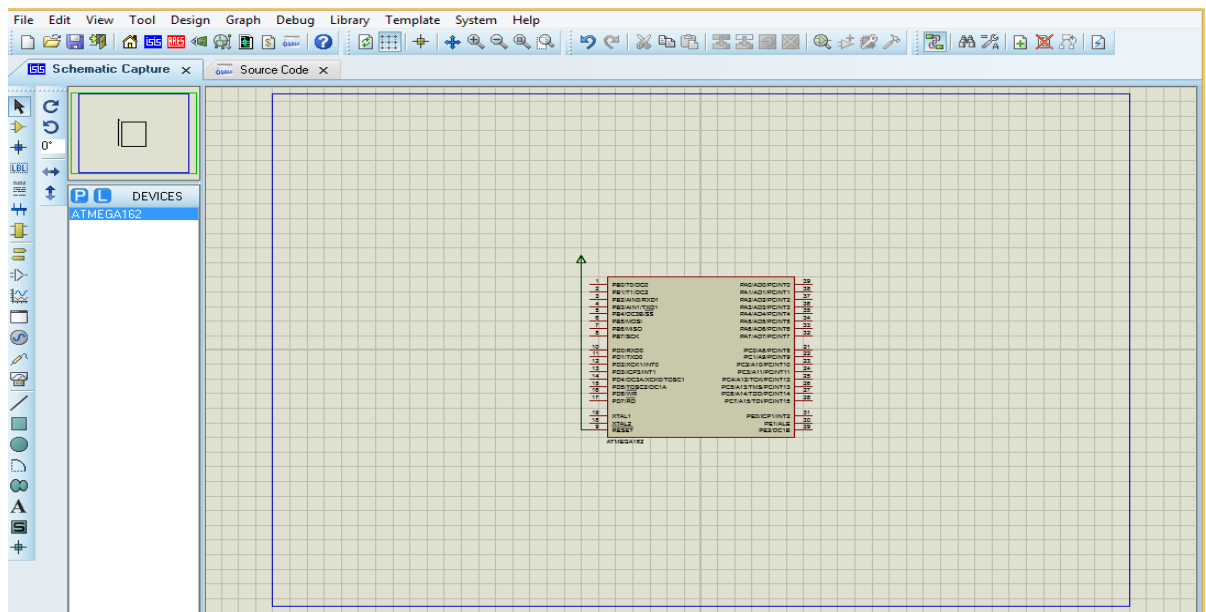
```

    }
}

```

Procedure

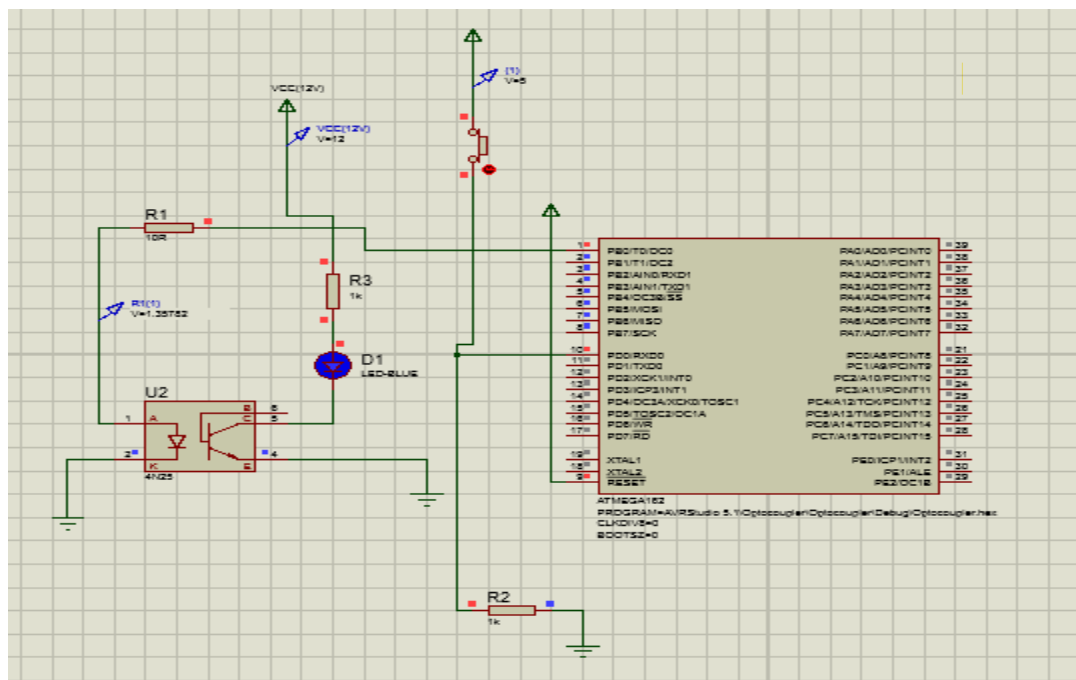
1. Launch Atmel Studio. Create new Project by selecting Atmega162 as device.
2. Write the given source code and compile to create hex file.
3. Now launch Proteus Design Suite select new project and write name of your project. Click next.
4. On next page, select create a schematic from selected template and select default. Click next.
5. Select create firmware project, choose AVR as family controller as Atmega162 and click finish.
6. From the menu bar select ISIS schematic capture. Schematic capture window will open with Atmega162 added.



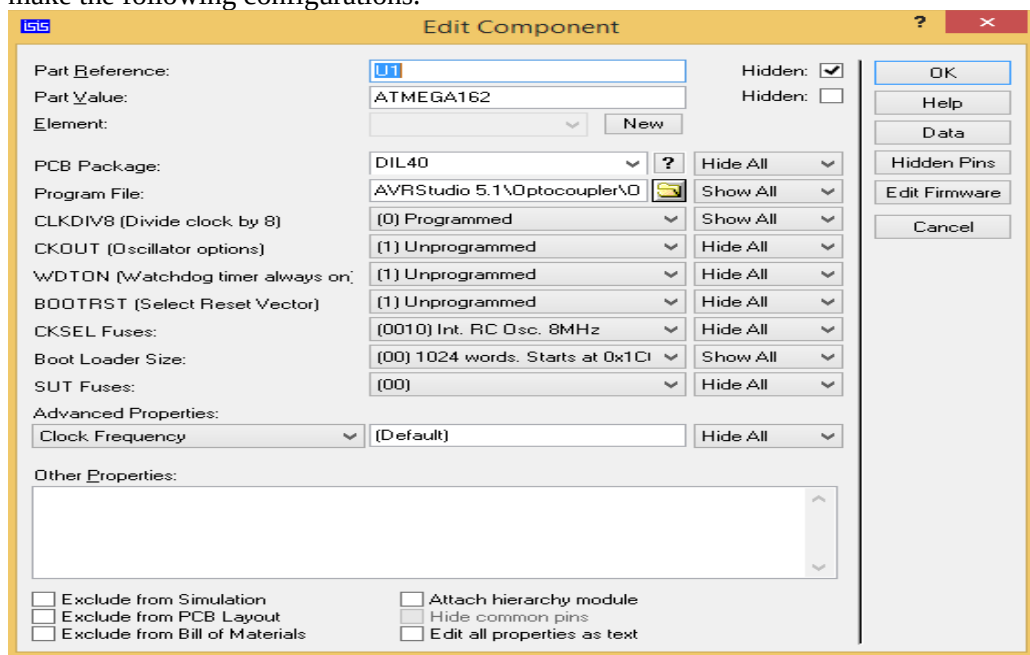
7. From the leftmost pane select component mode, click on P Under Devices to pick all the components to be added on the schematic.
8. Add optocoupler 4N25, resistances two 1K and one 10 ohm , LED- blue, push button and place all the components on workspace.



9. From the leftmost pane select terminals mode and select power and ground terminals. You can go to design menu, configure power rails to change the power from 5V to 12V. We need two 5V and one 12V power rail.
10. Wire together all the components as shown in the schematic below.



11. Now double click on the microcontroller. An Edit Component dialogue box will open where make the following configurations.



12. Browse in Program file to select the hex file created earlier and click ok.
13. Now run the simulation from the bottom bar by clicking on play button.



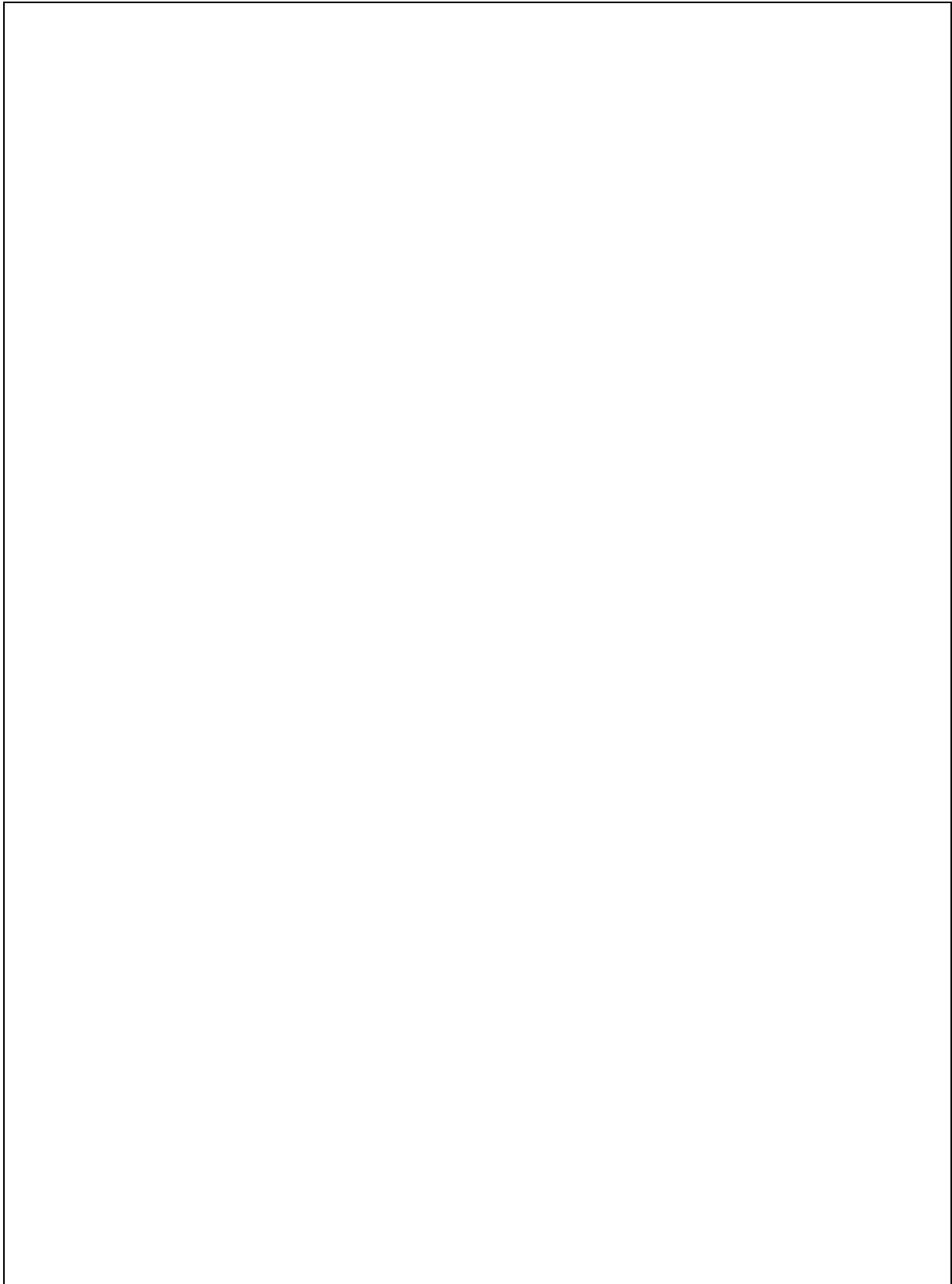
Briefly describe how the simulation works and note down the separation of voltages.

[illegible]

1. Practice using opto-coupler in some microcontroller based circuit where sub-circuits are operating at different voltage levels.

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for writing or drawing. There are no margins, text, or other markings on the page.

Attach screenshots of your simulation here



Lab Session 15

Open Ended Lab

Problem statement

Complex problem solving attributes covered

Task description

Deliverables

Grading rubric

Solution

NED University of Engineering & Technology
Department of Computer and Information Systems Engineering
 Course Code and Title: CS-301 Microprocessor Based System Design



Laboratory Session No. _____

Date: _____

Psychomotor Domain Assessment Rubric-Level P3

Skill Sets	Extent of Achievement				
	0	1	2	3	4
<u>Equipment Identification</u> Sensory skill to <i>identify</i> equipment and/or its component for a lab work.	Not able to identify the equipment.	--	--	--	Able to identify equipment as well as its components.
<u>Equipment Use</u> Sensory skills to <i>demonstrate</i> the use of the equipment for the lab work.	Doesn't demonstrate the use of equipment.	Slightly demonstrates the use of equipment.	Somewhat demonstrates the use of equipment.	Moderately demonstrates the use of equipment.	Fully demonstrates the use of equipment.
<u>Procedural Skills</u> <i>Displays</i> skills to act upon sequence of steps in lab work.	Not able to either learn or perform lab work procedure.	Able to slightly understand lab work procedure and perform lab work.	Able to somewhat understand lab work procedure and perform lab work.	Able to moderately understand lab work procedure and perform lab work.	Able to fully understand lab work procedure and perform lab work.
<u>Response</u> Ability to <i>imitate</i> the lab work on his/her own.	Not able to imitate the lab work.	Able to slightly imitate the lab work.	Able to somewhat imitate the lab work.	Able to moderately imitate the lab work.	Able to fully imitate the lab work.
<u>Observation's Use</u> <i>Displays</i> skills to use the observations from lab work for experimental verifications and illustrations.	Not able to use the observations from lab work for experimental verifications and illustrations.	Slightly able to use the observations from lab work for experimental verifications and illustrations.	Somewhat able to use the observations from lab work for experimental verifications and illustrations.	Moderately able to use the observations from lab work for experimental verifications and illustrations.	Fully able to use the observations from lab work for experimental verifications and illustrations.
<u>Safety Adherence</u> Adherence to <i>safety</i> procedures.	Doesn't adhere to safety procedures.	Slightly adheres to safety procedures.	Somewhat adheres to safety procedures.	Moderately adheres to safety procedures.	Fully adheres to safety procedures.
<u>Equipment Handling</u> <i>Equipment care</i> during the use.	Doesn't handle equipment with required care.	Rarely handles equipment with required care.	Occasionally handles equipment with required care.	Often handles equipment with required care.	Handles equipment with required care.
<u>Group Work</u> <i>Contributes</i> in a group based lab work.	Doesn't participate and contribute.	Slightly participates and contributes.	Somewhat participates and contributes.	Moderately participates and contributes.	Fully participates and contributes.

Weighted CLO (Psychomotor Score)

Remarks

Instructor's Signature with Date:

NED University of Engineering & Technology
Department of Computer and Information Systems Engineering



Course Code and Title: CS-301 Microprocessor Based System Design

Laboratory Session No. _____

Date: _____

Psychomotor Domain Assessment Rubric-Level P3					
Skill Sets	Extent of Achievement				
	0	1	2	3	4
<u>Equipment Identification</u> Sensory skill to <i>identify</i> equipment and/or its component for a lab work.	Not able to identify the equipment.	--	--	--	Able to identify equipment as well as its components.
<u>Equipment Use</u> Sensory skills to <i>demonstrate</i> the use of the equipment for the lab work.	Doesn't demonstrate the use of equipment.	Slightly demonstrates the use of equipment.	Somewhat demonstrates the use of equipment.	Moderately demonstrates the use of equipment.	Fully demonstrates the use of equipment.
<u>Procedural Skills</u> <i>Displays</i> skills to act upon sequence of steps in lab work.	Not able to either learn or perform lab work procedure.	Able to slightly understand lab work procedure and perform lab work.	Able to somewhat understand lab work procedure and perform lab work.	Able to moderately understand lab work procedure and perform lab work.	Able to fully understand lab work procedure and perform lab work.
<u>Response</u> Ability to <i>imitate</i> the lab work on his/her own.	Not able to imitate the lab work.	Able to slightly imitate the lab work.	Able to somewhat imitate the lab work.	Able to moderately imitate the lab work.	Able to fully imitate the lab work.
<u>Observation's Use</u> <i>Displays</i> skills to use the observations from lab work for experimental verifications and illustrations.	Not able to use the observations from lab work for experimental verifications and illustrations.	Slightly able to use the observations from lab work for experimental verifications and illustrations.	Somewhat able to use the observations from lab work for experimental verifications and illustrations.	Moderately able to use the observations from lab work for experimental verifications and illustrations.	Fully able to use the observations from lab work for experimental verifications and illustrations.
<u>Safety Adherence</u> Adherence to <i>safety</i> procedures.	Doesn't adhere to safety procedures.	Slightly adheres to safety procedures.	Somewhat adheres to safety procedures.	Moderately adheres to safety procedures.	Fully adheres to safety procedures.
<u>Equipment Handling</u> <i>Equipment care</i> during the use.	Doesn't handle equipment with required care.	Rarely handles equipment with required care.	Occasionally handles equipment with required care.	Often handles equipment with required care.	Handles equipment with required care.
<u>Group Work</u> <i>Contributes</i> in a group based lab work.	Doesn't participate and contribute.	Slightly participates and contributes.	Somewhat participates and contributes.	Moderately participates and contributes.	Fully participates and contributes.

Weighted CLO (Psychomotor Score)

Remarks

Instructor's Signature with Date:

NED University of Engineering & Technology
Department of Computer and Information Systems Engineering



Course Code and Title: CS-301 Microprocessor Based System Design

Laboratory Session No. _____

Date: _____

Psychomotor Domain Assessment Rubric-Level P3					
Skill Sets	Extent of Achievement				
	0	1	2	3	4
<u>Equipment Identification</u> Sensory skill to <i>identify</i> equipment and/or its component for a lab work.	Not able to identify the equipment.	--	--	--	Able to identify equipment as well as its components.
<u>Equipment Use</u> Sensory skills to <i>demonstrate</i> the use of the equipment for the lab work.	Doesn't demonstrate the use of equipment.	Slightly demonstrates the use of equipment.	Somewhat demonstrates the use of equipment.	Moderately demonstrates the use of equipment.	Fully demonstrates the use of equipment.
<u>Procedural Skills</u> <i>Displays</i> skills to act upon sequence of steps in lab work.	Not able to either learn or perform lab work procedure.	Able to slightly understand lab work procedure and perform lab work.	Able to somewhat understand lab work procedure and perform lab work.	Able to moderately understand lab work procedure and perform lab work.	Able to fully understand lab work procedure and perform lab work.
<u>Response</u> Ability to <i>imitate</i> the lab work on his/her own.	Not able to imitate the lab work.	Able to slightly imitate the lab work.	Able to somewhat imitate the lab work.	Able to moderately imitate the lab work.	Able to fully imitate the lab work.
<u>Observation's Use</u> <i>Displays</i> skills to use the observations from lab work for experimental verifications and illustrations.	Not able to use the observations from lab work for experimental verifications and illustrations.	Slightly able to use the observations from lab work for experimental verifications and illustrations.	Somewhat able to use the observations from lab work for experimental verifications and illustrations.	Moderately able to use the observations from lab work for experimental verifications and illustrations.	Fully able to use the observations from lab work for experimental verifications and illustrations.
<u>Safety Adherence</u> Adherence to <i>safety</i> procedures.	Doesn't adhere to safety procedures.	Slightly adheres to safety procedures.	Somewhat adheres to safety procedures.	Moderately adheres to safety procedures.	Fully adheres to safety procedures.
<u>Equipment Handling</u> <i>Equipment care</i> during the use.	Doesn't handle equipment with required care.	Rarely handles equipment with required care.	Occasionally handles equipment with required care.	Often handles equipment with required care.	Handles equipment with required care.
<u>Group Work</u> <i>Contributes</i> in a group based lab work.	Doesn't participate and contribute.	Slightly participates and contributes.	Somewhat participates and contributes.	Moderately participates and contributes.	Fully participates and contributes.

Weighted CLO (Psychomotor Score)

Remarks

Instructor's Signature with Date: